



西安郵電大學

毕业设计（论文）

题目：基于容器的云文件管理系统设计与实现

学 院: 计算机学院

专 业: 网络工程

班 级: 2203

学 号: 04222088

学生姓名：杨佳星

导师姓名: 王晓梅 职称: 讲师

起止时间: 2025 年 12 月 9 日至 2026 年 6 月 5 日

2026 年 6 月 5 日

毕业设计（论文）承诺书

本人所提交的毕业设计（论文）《基于容器化部署的云文件管理系统设计与实现》是本人在指导教师指导下独立研究、写作的成果，毕业设计（论文）中所引用他人的文献、数据、图件、资料均已明确标注；对本文的研究做出重要贡献的个人和集体，均已在文中以明确方式注明并表示感谢。

本人深知本承诺书的法律责任，违规后果由本人承担。

论文作者签名：_____ 日 期：_____

关于毕业设计（论文）使用授权的声明

本人在导师指导下所完成的论文及相关的职务作品，知识产权归属西安邮电大学。本人完全了解西安邮电大学有关保存、使用毕业设计（论文）的规定，同意学校保存或向国家有关部门或机构送交论文的纸质版和电子版，允许论文被查阅和借阅；本人授权西安邮电大学可以将本毕业设计（论文）的全部或部分内容编入有关数据库进行检索，可以采用任何复制手段保存和汇编本毕业设计（论文）。本人离校后发表、使用毕业设计（论文）或与该毕业设计（论文）直接相关的学术论文或成果时，第一署名单位仍然为西安邮电大学。

本毕业设计（论文）研究内容：

☐可以公开

☐不宜公开，已办理保密申请，解密后适用本授权书。

（请在以上选项内选择其中一项打“√”）

论文作者签名：_____

导师签名：_____

日 期：_____

日 期：_____

西安邮电大学本科毕业设计(论文)选题审批表

申报人	王晓梅	职 称	讲师	学 院	计算机学院
题目名称	基于容器的云文件管理系统设计与实现				
题目来源	☐ 教师科研课题 ☒ 教师专业实践 ☐ 其他				
题目类型	☐ 艺术作品 ☐ 硬件设计 ☒ 软件设计 ☐ 论文				
题目分类	☒ 工程实践 ☐ 社会调查 ☐ 实习 ☐ 实验 ☐ 其他				
题目简述	(为什么申报该课题) 随着企业数字化办公的深入推进，传统文件管理系统在应对海量文件存储、多终端协同及高并发访问时暴露出显著短板：本地化部署模式导致存储成本年均增长25%，且跨区域访问延迟高达3秒以上；文件版本混乱、权限管控粗放等问题，造成企业数据泄露风险逐年提升；在远程办公、在线教育等高并发场景下，系统崩溃率年年增长，严重影响业务连续性。特别是在媒体制作、科研计算等文件规模超PB级的企业中，传统文件管理系统的年度维护成本占IT预算比例越来越高，且功能扩展周期长，难以满足企业敏捷运营需求。 本课题拟采用容器化技术，设计并实现一套具备高弹性扩展与强安全管控的云文件管理系统。系统采用容器化部署方案，通过自动扩缩容，确保在高并发上传/下载场景下仍能保持快速稳定响应；利用容器镜像的标准化封装特性，将文件存储、转码处理、权限管控等核心模块进行统一封装，彻底消除开发、测试、生产环境的部署差异，实现"一次构建，全域部署"的敏捷交付模式，有效支撑企业数字化文件管理需求。				
对学生知识与能力要求	1. 熟练使用个人计算机，熟悉相关计算机基础知识； 2. 熟练掌握编程技能； 3. 具备一定Web应用前后端开发能力； 4. 具有一定的专业文献阅读及翻译能力； 5. 良好的创新思维和问题解决能力。				

具体任务以及预期目标	本课题期望设计与实现基于容器的云文件管理系统。预期目标是： 1. 搭建系统的开发环境； 2. 根据课题进行需求分析； 3. 实现文件存储管理功能； 4. 实现转码处理功能； 5. 创建权限管控像； 6. 进行测试； 7. 培养学生独立自主的学习能力并提高学生的设计开发能力。		
时间进度	1. 2025 年 12 月 9 日-2025 年 12 月 16 日 学习相关知识，掌握基本原理，撰写开题报告提交系统； 2. 2025 年 12 月 17 日-2026 年 01 月 20 日 熟悉所要开发的系统并进行系统分析； 3. 2026 年 01 月 21 日-2026 年 02 月 10 日 进行概要设计，详细设计； 4. 2026 年 02 月 11 日-2026 年 03 月 21 日 根据设计进行环境搭建，软件编码并实现，进行中期检查； 5. 2026 年 03 月 22 日-2026 年 05 月 10 日 测试程序，分析结果，进行迭代，进行第一次验收； 6. 2026 年 05 月 11 日-2026 年 06 月 05 日 终期验收，论文撰写、答辩。		
专业负责人审核意见	签字：年 月 日		
系（教研室）主任 签字	年 月 日	主管院长 签字	年 月 日

西安邮电大学本科毕业设计（论文）开题报告

学生姓名	杨佳星	学号	04222088	专业班级	网络 2203
指导教师	王晓梅	题目	基于容器的云文件管理系统设计与实现		

1、选题目的（为什么选该课题）

随着企业数字化转型的深入推进和云计算技术的快速发展，文件数据呈爆炸式增长，企业对文件管理系统的需求日益迫切。根据国际数据公司（IDC）的研究报告，全球数据总量正以每年约 25% 的速度增长，预计到 2025 年将达到 175ZB，其中非结构化数据（如文档、图片、视频等文件）占比超过 80%，在这一背景下如何高效、安全地管理海量文件数据成为企业信息化建设面临的重要挑战。

传统本地化部署的文件管理系统在存储扩展性、多终端协同访问、系统运维效率等方面暴露出明显短板，导致资源利用率低、流程僵化与系统扩展性不足。

首先，在存储扩展性方面，传统系统多采用集中式存储和单体应用架构，存储容量受限于物理硬件，数据量快速增长时往往需要停机扩容，影响业务连续性并抬升成本。

其次，在多终端协同方面，现代办公场景需要 PC、笔记本、平板和手机等多种设备访问文件，缺乏统一的跨平台能力会造成同步困难与版本管理混乱，严重影响工作效率。

在运维效率方面，传统系统部署和升级复杂，容易出现“在我机器上能跑”的环境依赖问题，系统迁移和灾备也面临较大挑战，运维成本居高不下。最后，在访问性能方面，跨地域访问延迟较高且缺乏弹性伸缩能力，业务高峰期容易出现响应缓慢甚至服务中断的情况。

容器技术（如 Docker）及其编排平台（如 Docker Compose、Kubernetes）在云计算领域迅速发展，为构建弹性、可扩展的云原生应用提供了坚实技术基础，形成了“构建一次、运行全域”的能力^[1]。

Docker 通过将应用及其依赖打包成标准化容器镜像，实现应用与运行环境解耦，具有环境一致性、快速部署、资源隔离与弹性伸缩等显著优势^{[2][3]}。将文件存储、转码处理、用户认证等模块以容器方式封装并结合编排技术，可有效降低环境差异带来的部署问题，提升系统可移植性与运维效率，实现模块级弹性扩展。系统的后端实现选择以 Spring Boot 为基础的快速开发框架，可降低工程搭建成本并贴合主流前后端分离实践^[4]。在此背景下，设计并实现一套基于容器的云文件管理系统，不仅有助于提高系统扩展性、可靠性与安全性、降低运维成本，还能帮助网络工程专业学生系统掌握云原生技术体系，提升综合实践能力。

2、国内外研究现状

（1）云存储与云文件管理系统研究现状

在国际上，云存储服务已形成较为成熟的产品体系，Dropbox、Google Drive、Microsoft OneDrive 与 Amazon S3 等平台依托分布式存储、CDN 与多级缓存技术提供高可靠性与高性能的文件访问服务，为云文件管理奠定了实践基础。国内方面，阿里云 OSS、腾讯云 COS、华为云 OBS 等对象存储服务在互联网、金融与政务行业得到广泛应用，同时 NextCloud、Seafile 等开源网盘系统也在私有部署场景获得验证。

（2）容器化与微服务架构的应用现状

容器化与微服务架构已经成为现代应用系统的重要技术路线，Docker、Kubernetes 与 Docker Compose 为多服务协同、自动扩缩容和持续交付提供了标准化支撑^{[5][6][7]}。在文件服务与媒体处理领域，越来越多的系统尝试将文件上传、转码、缩略图生成等功能拆分成独立容器服务，以获得更好的模块解耦与弹性扩展能力。

（3）多媒体文件处理技术研究

多媒体处理方面，FFmpeg 等开源工具可实现音视频转码与格式标准化，结合文档转 PDF 与在线渲染技术能够满足多类型文件的在预览需求，为云文件管理系统的多格式适配提供支

撑。

(4) 用户认证与安全技术研究

在身份认证与访问控制方面，JSON Web Token (JWT) 等轻量级方案可在前后端分离架构下实现无状态认证，配合安全加密策略能够保障系统访问安全性与可扩展性^{[8][9]}。

(5) 现有研究的不足与本课题的切入点

现有云文件管理系统虽在工程实践上较为成熟，但在教学示范、容器化一体化集成和完整工程流程描述方面仍存在不足，因此本课题拟通过标准化容器封装与编排展示从单体应用向云原生系统演进的路径，为工程实践与教学提供参考。

参考文献：

- [1] Merkel D. Docker: lightweight linux containers for consistent development and deployment[J]. Linux journal, 2014, 2014(239): 2.
- [2] Pahl C, Brogi A, Soldani J, et al. Cloud container technologies: a state-of-the-art review[J]. IEEE Transactions on Cloud Computing, 2019, 7(3): 677-692.
- [3] Turnbull J. The Docker Book: Containerization is the new virtualization[M]. James Turnbull, 2014.
- [4] 李刚, 张华. Spring Boot 实战[M]. 北京: 人民邮电出版社, 2020.
- [5] Kratzke N, Quint P C. Understanding cloud-native applications after 10 years of cloud computing-A systematic mapping study[J]. Journal of Systems and Software, 2017, 126: 1-16.
- [6] 刘超. Kubernetes 权威指南: 从 Docker 到 Kubernetes 实践全接触[M]. 北京: 电子工业出版社, 2021.
- [7] Dragoni N, Giallorenzo S, Lafuente A L, et al. Microservices: yesterday, today, and tomorrow[M]. Present and ulterior software engineering. Springer, Cham, 2017: 195-216.
- [8] 王晓峰, 刘洋. 云存储系统中的数据安全与隐私保护研究综述[J]. 计算机研究与发展, 2020, 57(10): 2072-2089.
- [9] Jones M, Bradley J, Sakimura N. JSON web token (JWT)[S]. RFC 7519, 2015.

前期基础（已学课程、掌握的工具，资料积累、软硬件条件等）

1. 已学课程：

计算机网络、数据结构、操作系统、Java 程序设计、数据库原理、Web 开发技术

2. 掌握工具：

IntelliJ IDEA、Visual Studio Code、PyCharm、Docker Desktop

3. 资料积累：

- (1) 阅读了 Docker 官方文档及相关技术博客，了解容器化部署的基本流程；
- (2) 学习了 Spring Boot 官方指南，掌握 RESTful API 开发方法；
- (3) 研读了多篇关于云存储系统设计的学术论文，了解业界最佳实践；

(4) 参考了开源文件管理系统的设计思路和实施方案。

4. 软硬件条件:

- (1) 硬件: 个人电脑 (8GB 以上内存, 支持虚拟化技术);
- (2) 操作系统: Windows 11 / macOS, 支持 WSL2 或 Docker Desktop;
- (3) 开发环境: JDK 17、Node.js 18、MySQL 8.0、Redis 7.x;
- (4) 容器环境: Docker Engine、Docker Compose。

要研究和解决的问题 (做什么)

本课题拟研究和解决的核心问题包括:

1、系统架构设计问题

如何设计一套支持高并发访问与弹性扩展的云文件管理系统架构? 需要解决前后端分离架构下的接口设计、数据库设计、缓存策略等问题, 确保系统具备良好的可扩展性和可维护性。具体而言, 需要设计合理的系统分层架构, 明确各层职责边界; 需要设计 RESTful 风格的 API 接口, 保证接口的规范性和易用性; 需要设计合理的数据库表结构, 支持文件元数据的高效存储和查询; 需要引入 Redis 缓存机制, 减轻数据库压力, 提升系统响应速度。

2、文件存储与管理问题

如何实现高效的文件上传、下载、删除、目录管理等核心功能? 需要设计合理的文件元数据管理机制和存储路径映射策略, 建立文件物理存储与逻辑目录结构的映射关系。需要支持大文件分片上传功能, 将大文件切分为多个小块分别上传, 支持断点续传, 解决大文件上传过程中的超时和失败问题。需要实现秒传功能, 通过 MD5 哈希值校验判断文件是否已存在, 避免重复上传, 提高文件操作的效率和用户体验。同时需要实现回收站功能, 支持文件的软删除和恢复, 防止用户误删重要文件。

3、媒体文件转码与预览问题

如何实现音视频文件和文档的在线转码处理? 需要集成 FFmpeg 等转码工具, 实现视频格式转换功能, 将各种格式的视频转换为 Web 友好的 MP4/H.264 格式。需要设计转码任务队列和状态管理机制, 支持异步转码处理。需要实现多种格式文件的在线预览功能, 包括视频、音频、图片、PDF 等常见格式, 提升用户体验。此外, 还需实现视频和文档的水印功能, 在文件上添加用户标识或版权信息, 保护文件版权和追踪文件来源。

4、用户认证与文件分享问题

如何实现安全可靠的用户认证机制? 拟采用 JWT (JSON Web Token) 实现无状态认证, 用户登录成功后服务端签发 Token, 后续请求携带 Token 进行身份验证, 无需在服务端保存会话状态, 便于系统的水平扩展。密码使用 BCrypt 算法加密存储, 通过加盐和多轮哈希有效防止密码泄露。同时需要实现文件分享功能, 支持生成分享链接和提取码, 用户可以将文件分享给他人访问; 支持设置分享有效期, 过期后分享链接自动失效, 保障文件安全。

5、容器化部署问题

如何将文件服务、转码服务、数据库等组件进行容器化封装和编排? 需要为各个服务组件编写 Dockerfile, 定义镜像构建过程, 采用多阶段构建优化镜像体积。需要编写 docker-compose.yml 配置文件, 定义多个服务容器的编排关系、网络配置、数据卷挂载等。需要配置 Nginx 作为反向代理, 统一前后端访问入口, 实现请求转发和负载均衡。需要通过环境变量实现配置与代码分离, 便于在不同环境下部署。最终实现系统的一键部署和多服务协同运行, 降低部署和运维成本。

6、系统性能优化问题

在有限硬件资源条件下, 如何通过合理设计与优化保证系统在一定并发访问量下的稳定性与性能? 需要引入 Redis 缓存机制, 缓存热点数据和会话信息, 减少数据库查询次数。需

要优化数据库连接池配置，提高数据库连接的复用率。需要采用异步处理技术，将耗时操作（如文件转码）放入消息队列异步执行，避免阻塞主线程。需要实现存储空间统计功能，实时显示用户已用空间和总空间，便于用户了解空间使用情况，合理管理文件存储。

工作思路和方案（怎么做）

1、总体技术路线

本课题拟采用"需求分析→系统设计→模块实现→容器化部署→测试与分析"的整体技术路线进行研究。首先通过对典型企业文件管理场景的调研，梳理系统功能需求和非功能需求，明确系统边界和核心功能；然后进行系统总体架构设计、模块划分和数据库设计，绘制系统架构图和 ER 图；接着按照设计文档分模块实现各项功能，采用迭代开发方式逐步完善；最后进行容器化封装、系统集成测试和性能测试，验证系统的功能正确性和性能表现。

2、系统架构设计方案

系统采用前后端分离的 B/S 架构，整体架构分为表现层、业务逻辑层、数据访问层三层，各层之间通过接口进行解耦，便于独立开发和维护。

前端使用 Vue 3 框架构建单页应用（SPA），采用组件化开发模式，使用 Element Plus 组件库提供丰富的 UI 组件，实现美观、易用的用户界面。通过 Axios 与后端 API 进行 HTTP 通信，使用 Pinia 进行全局状态管理，使用 Vue Router 实现前端路由。前端构建使用 Vite 作为构建工具，支持热模块替换，提升开发效率。

后端使用 Spring Boot 3.x 框架构建 RESTful API 服务，采用 Controller-Service-Mapper 三层架构。Controller 层负责接收和响应 HTTP 请求，Service 层实现业务逻辑，Mapper 层通过 MyBatis-Plus 与数据库交互。使用 Spring Security 进行安全控制，集成 JWT 实现身份认证。

数据存储层采用 MySQL 8.0 作为关系型数据库，存储用户信息、文件元数据、分享记录等业务数据。采用 Redis 7.x 作为缓存数据库，缓存用户 Token、热点数据等，提升访问性能。文件实体存储在服务器本地文件系统，通过数据卷挂载实现持久化。

通过 Nginx 作为反向代理服务器，监听 80 端口，将/api 开头的请求转发到后端服务，其他请求返回前端静态文件，实现前后端的统一访问入口。

3、核心功能模块设计

（1）文件管理模块：实现文件上传（支持分片上传和 MD5 秒传）、下载、删除、重命名、移动、复制等功能。设计 file_info 表存储文件元数据，包括文件名、大小、类型、MD5 值、存储路径、创建时间等字段。实现回收站功能，删除的文件先移入回收站，支持恢复或彻底删除。实现文件夹管理功能，支持创建、重命名、删除文件夹，支持多级目录结构。

（2）转码处理模块：集成 FFmpeg 实现音视频转码，支持将各种格式的视频转换为 MP4 格式，便于在线播放。实现视频、音频、图片、PDF 等多种格式文件的在线预览功能。实现视频和文档的水印功能，支持添加文字水印或图片水印，保护文件版权。

（3）用户认证模块：实现用户注册功能，包括用户名、密码、邮箱等信息的录入和校验。实现用户登录功能，采用 JWT 进行身份认证，Token 有效期设置为 24 小时，支持自动刷新。密码使用 BCrypt 算法加密存储，强度因子设置为 10。实现存储空间统计功能，显示用户已用空间和总空间配额。

（4）文件分享模块：实现文件分享链接生成功能，为分享的文件生成唯一的分享码。支持设置分享有效期，可选择 1 天、7 天、30 天或永久有效。实现分享链接的访问验证，输入正确的提取码后可下载或预览文件。支持查看分享记录和取消分享。

（5）文件搜索模块：实现文件名模糊搜索功能，支持用户快速定位文件。搜索结果按相关度和时间排序，提供良好的搜索体验。

4、容器化部署方案

采用 Docker 和 Docker Compose 实现系统的容器化部署，整体部署架构包括 5 个服务容器：

(1) nginx 容器：基于 nginx:alpine 镜像，作为反向代理和静态资源服务器，监听 80 端口。

(2) frontend 容器：基于 Node 镜像构建前端应用，采用多阶段构建，最终镜像基于 nginx:alpine，仅包含构建产物。

(3) backend 容器：基于 Maven 镜像构建后端应用，采用多阶段构建，最终镜像基于 eclipse-temurin:17-jre，仅包含 JAR 包和 JRE 运行时。

(4) mysql 容器：基于 mysql:8.0 镜像，通过数据卷挂载实现数据持久化，初始化时自动执行建表脚本。

(5) redis 容器：基于 redis:7-alpine 镜像，提供缓存服务。

通过 docker-compose.yml 文件定义服务编排，配置容器间的网络通信（使用自定义 bridge 网络）、数据卷挂载、环境变量注入、健康检查和启动依赖顺序。编写 deploy.sh 部署脚本，实现一键构建镜像和启动所有服务。部署方案支持在 Windows WSL2、macOS 或 Linux 服务器环境下运行。

3. 进度计划

(1) 2025 年 12 月 9 日-2025 年 12 月 16 日 学习相关知识，掌握基本原理，撰写开题报告提交系统；

(2) 2025 年 12 月 17 日-2026 年 01 月 20 日 熟悉所要开发的系统并进行系统分析；

(3) 2026 年 01 月 21 日-2026 年 02 月 10 日 进行概要设计，详细设计；

(4) 2026 年 02 月 11 日-2026 年 03 月 21 日 根据设计进行环境搭建，软件编码并实现，进行中期检查；

(5) 2026 年 03 月 22 日-2026 年 05 月 10 日 测试程序，分析结果，进行迭代，进行第一次验收；

(6) 2026 年 05 月 11 日-2026 年 06 月 05 日 终期验收，论文撰写、答辩。

指导教师意见

签字：

年 月 日

西安邮电大学毕业设计（论文）成绩评定表

学生姓名	杨佳星	性别	男	学号	04222088	专业班级	网络工程		
	2203								
课题名称	基于容器的云文件管理系统设计与实现								
指导教师意见	支撑指标点/赋分	3. 3/10	3. 4/10	5. 1/20	5. 2/20	5. 3/10	5. 4/10	11. 1/20	合计
	得分								
	指导教师(签字): 年 月 日								
评阅教师意见	支撑指标点/赋分	3. 3/20		3. 4/20		5. 1/30		11. 1/30	合计
	得分								
	评阅教师(签字): 年 月 日								
验收小组意见	支撑指标点/赋分	3. 3/10	3. 4/10	5. 1/10	5. 3/20	5. 4/30	11. 1/20	合计	
	得分								
	验收小组组长(签字): 年 月 日								
答辩小组意见	支撑指标点/赋分	3. 3/10		3. 4/10		5. 1/20		11. 1/60	合计
	得分								
	答辩小组组长(签字): 年 月 日								
学生总评成绩	评分比例	指导教师(20%)		评阅教师(30%)		验收小组(20%)		答辩小组(30%)	合计
	评分								
	毕业论文(设计)最终等级制成绩(优秀、良好、中等、及格、不及格)								
答辩委员会意见	学院答辩委员会主任(签字、学院盖章): 年 月 日								

摘 要

高校课程设计、协作开发与日常学习场景中，电子文件的存储、共享与预览需求逐渐增多，本地文件夹与个人云盘在多人协作和长期归档中存在分散、不易复现的不足。本文设计并实现了一套基于容器的轻量级云文件管理系统，围绕真实可部署、可演示、可复用的目标展开。系统采用前后端分离架构，前端基于 Vue 3 与 Element Plus 构建，后端采用 Spring Boot 3 配合 MyBatis-Plus 实现接口与数据访问，元数据存储于 MySQL，登录态与上传临时状态借助 Redis 维护，并通过 Nginx 提供统一访问入口；整体由 Docker Compose 编排 lfs-nginx、lfs-backend、lfs-mysql、lfs-redis 四个核心容器，借助宿主机目录完成文件、数据库与日志的持久化。在功能层面，系统覆盖用户认证、文件与目录管理、原文件与封面预览、分享码访问、回收站及存储统计；在隔离层面，以 lfs_file.user_id 字段配合统一的当前用户解析服务，实现普通用户与管理员的视角区分；在预览层面，针对反向代理后路径前缀异常导致的预览失败问题，对路径归一化与 secret + expire 签名校验进行了修复。测试结果表明，系统在 Docker Compose 环境下可一键部署，登录、上传、预览、分享、回收站及双用户隔离等功能运行稳定，能够满足毕业设计演示与课程协作场景的实际使用需求。

关键词：云文件管理；容器化部署；Docker Compose；Spring Boot；用户隔离

ABSTRACT

With the continuous expansion of digital learning, office work, and project collaboration scenarios, the scale of file data has been growing steadily. Traditional file management methods have gradually exposed shortcomings in deployment complexity, environmental dependency, online preview stability, and sharing convenience. To address these problems, this paper designs and implements a cloud file management system based on containerized deployment. The system adopts a front-end and back-end separation architecture. The front end is built with Vue 3 and Element Plus to provide the user interaction interface, while the back end is implemented with Spring Boot, MyBatis-Plus, MySQL, and Redis to support business logic and data management. In addition, Nginx and Docker Compose are used to form a unified deployment solution, enabling rapid system construction and one-click startup.

Focusing on the core business of file management, the system implements user authentication, file upload, directory management, file preview, file sharing, recycle bin management, and storage statistics. In response to the original file preview and cover preview failures found during the final stage of the project, this paper repairs and optimizes the preview access chain, thereby improving the stability of file access. To satisfy the requirements of graduation project demonstration and practical use, the system further introduces a lightweight two-user isolation mechanism. By combining file ownership fields with a current-user identification service, data isolation between the administrator and the ordinary demo user is achieved. Meanwhile, database compatibility patches are applied to ensure smooth upgrading in legacy data environments.

Test results show that the system can be rapidly deployed through Docker Compose, and that its core functions, including login, upload, preview, sharing, recycle bin operations, and user isolation, run stably. Therefore, the system has good engineering practice value and demonstrative significance for course design and graduation project development.

Key words: Cloud file management system; Containerized deployment; Docker Compose;
File preview; User isolation

目 录

第 1 章 绪论	1
1.1 研究背景.....	1
1.2 国内外研究现状.....	2
1.2.1 国外研究与产品现状	2
1.2.2 国内研究与产品现状	2
1.2.3 现存问题与本课题切入点	3
1.3 研究内容.....	3
1.4 论文组织结构.....	4
第 2 章 基础知识与相关技术	5
2.1 前端技术.....	5
2.1.1 Vue 3	5
2.1.2 Element Plus	5
2.1.3 Vite 与 Vue Router	6
2.2 后端技术.....	6
2.2.1 Spring Boot	6
2.2.2 MyBatis-Plus	6
2.2.3 安全机制与支撑库	7
2.3 数据存储技术.....	7
2.3.1 MySQL	7
2.3.2 Redis	7
2.4 部署技术.....	8
2.4.1 Nginx	8
2.4.2 Docker 与 Docker Compose	8
2.5 文件处理与预览技术.....	9
2.5.1 文档转换与 PDF 渲染	9
2.5.2 音视频处理	9
2.5.3 本地文件访问与签名防盗链	9
2.6 本章小结.....	10
第 3 章 概要设计	11
3.1 需求分析.....	11
3.1.1 功能需求	11
3.1.2 非功能需求	11

3.2	设计目标与系统边界.....	12
3.3	总体架构设计.....	12
3.3.1	层次结构.....	12
3.3.2	网络访问关系.....	13
3.4	功能模块划分.....	14
3.4.1	模块组织.....	14
3.4.2	模块协作时序.....	16
3.5	数据库设计.....	16
3.5.1	核心表结构.....	16
3.5.2	双用户隔离的核心字段.....	16
3.6	接口设计.....	17
3.6.1	REST 接口分组.....	17
3.6.2	预览访问与普通路由的分离.....	17
3.7	本章小结.....	18
第4章	详细设计与系统实现.....	19
4.1	用户认证模块实现.....	19
4.1.1	模块职责与接口划分.....	19
4.1.2	登录流程.....	19
4.1.3	当前用户解析服务.....	21
4.1.4	异常处理与登录态过期.....	21
4.2	文件管理与双用户隔离实现.....	22
4.2.1	上传与秒传实现.....	22
4.2.2	文件列表与双用户隔离.....	23
4.2.3	启动期数据库兼容补丁.....	24
4.2.4	目录树、移动与命名冲突处理.....	25
4.3	预览与分享模块实现.....	26
4.3.1	预览模块职责与访问链路.....	26
4.3.2	路径归一化的修复原因与实现.....	27
4.3.3	secret + expire 签名校验.....	27
4.3.4	分享模块的设计思路.....	27
4.3.5	预览异常修复回顾与回归用例.....	29
4.4	回收站与存储统计实现.....	29
4.4.1	回收站模块设计.....	29
4.4.2	按用户隔离的存储统计实现.....	30
4.4.3	模块视角一致性.....	31
4.5	容器化部署实现.....	32

4.5.1	部署组成	32
4.5.2	一键部署与数据持久化	33
4.5.3	配置项与环境变量	34
4.5.4	部署风险与处理方式	34
4.6	本章小结	34
第5章	测试和验证	35
5.1	测试目标与测试范围	35
5.1.1	测试目标	35
5.1.2	测试范围	35
5.2	测试环境	36
5.2.1	测试环境与约束	37
5.2.2	测试环境初始化	37
5.3	测试方案与测试数据	38
5.3.1	测试方案分层	38
5.3.2	测试数据准备	38
5.3.3	测试项与执行顺序	38
5.4	测试结果	39
5.4.1	自动化测试结果	41
5.4.2	运行表现观察	42
5.5	结果分析	43
5.5.1	容器化部署层面	43
5.5.2	系统各能力层面分析	43
5.5.3	测试结论	44
5.6	本章小结	44
	结论	46
	参考文献	47
	致谢	48

第 1 章 绪论

1.1 研究背景

近年来，云计算和容器技术发展得很快，Web 系统的部署方式也跟着变了不少。从早期物理机部署、虚拟机部署，到现在以容器为最小交付单元的轻量化方式，这条演进路线已经走得比较清楚。Docker、Docker Compose、Kubernetes 这一类编排工具，把环境差异这一长期存在的老问题压到比较低的水平^{[1][2][3]}。现在，它们已经是 Web 后端、文件存储、微服务系统比较常见的交付选择。云存储与云文件管理这类应用，在教学、办公、科研等领域的需求也一直在扩。功能范围早就不只是上传下载，而是延伸到目录组织、在线预览、链接分享、回收站管理、空间统计这一整条业务链。

在容器与云的关系层面，业界与学界已形成基本共识：以 Docker 为代表的容器技术对应用开发和部署两个方面都产生了积极影响，云架构设计正在向 DevOps 模式演进，支持持续开发与持续部署流水线，并逐渐采用基于容器及其编排机制的云原生架构方案，容器集群的管理与编排已经成为云平台的核心议题之一。

在本科毕业设计的语境下，文件管理类系统同时承担着两类要求：一是需将上传、预览、分享、回收、权限隔离等核心业务功能完整呈现，从而体现对前后端分离架构、数据库设计与中间件协同等综合知识的掌握程度；其二是在答辩环节中应可以快速恢复并稳定运行，以便进行现场演示。然而，传统的本地安装与人工部署方式，在依赖版本、静态资源路径、数据库初始化及文件目录权限等方面与运行环境耦合比较紧密，一旦更换部署主机或数据卷被清理，系统的重新启动往往需耗费较长时间。其结果为：功能实现难度并不高，但环境一致性较差，复现演示成本偏高，最终演示效果易受到环境因素的影响。

把系统装到容器里，在本课题里已经不是锦上添花的优化项，而是确保它能复现、能演示的前提。Docker 加 Docker Compose 提供的镜像构建和服务编排能力，让前端、后端、数据库、缓存这几个组件能装到一个不可变的运行单元里。再配合本地目录挂载做持久化，开发、测试、答辩演示三套环境就能维持在同一个状态。这种交付方式在轻量级 Web 系统里优势比较明显，最近几年高校实验平台、内部知识库这类场景用得也越来越多。

Pahl 等人对 46 篇容器技术研究文献的系统映射分析中指出，容器化是一种以轻量级方式实现应用虚拟化的技术，它通过从镜像仓库中获取单个镜像、并据此定制构建容器应用包，实现了相较于传统虚拟机更低的资源消耗与更快的启动速度，同时也能提供云中可移植、可互操作的软件应用所需的更高水平的应用封装能力^[4]。这一观察与本课题在毕业设计场景下对环境一致性、快速复现与跨主机演示的诉求高度吻合。

围绕上述需求，本文选题为“基于容器化部署的云文件管理系统设计与实现”，旨在实现一套面向轻量级演示场景的云文件管理平台。系统将在容器化部署、文件管理业务闭环、

预览访问链路修复及轻量级双用户隔离四个方面分别给出可运行的实现方案，并通过功能测试与部署验证对其在真实环境下的稳定性做了确认。云计算作为一种新兴的计算模式，通过互联网将计算资源以服务的方式提供给用户，具有按需自助服务、广泛网络接入、资源池化、快速弹性扩展和可计量服务五大基本特征^[5]。容器技术作为云计算 IaaS 与 PaaS 层之间的轻量虚拟化方案，凭借其镜像构建一次、跨平台运行的特性，已成为云原生应用最主流的交付方式之一。

1.2 国内外研究现状

1.2.1 国外研究与产品现状

在云存储与文件管理领域，国外起步相对较早。在商业产品方面，Dropbox、Google Drive、OneDrive 等系统已在文件同步、在线预览、链接分享与权限控制等基础能力上发展得较为成熟。在开源方案方面，Nextcloud、Seafile 在私有化部署、存储自主可控与二次扩展等方面也得到了广泛应用。在学术研究层面，国外文献主要关注四个方向^{[6][7]}：分布式与对象存储下的可扩展性与一致性问题；以签名 URL、零信任与细粒度访问控制为代表的文件分享安全机制；容器化与微服务架构下交付方式的轻量化研究；以及面向异构客户端的在线预览与转码优化。近年来，容器编排与对象存储相结合已成为云原生应用研究中的常用范式，相关综述与会议论文亦在持续补充新的成果。

从研究视角看，容器编排已被定义为对可能呈分布式部署的、基于容器的软件应用集群进行构建和持续管理的过程；它不仅负责容器的初始部署，还负责把多个容器作为一个整体来进行后续管理，包括可用性、伸缩性以及网络连接等方面^[4]。随着以 Docker 为代表的容器技术在 IaaS 与 PaaS 层面的广泛使用，云架构设计正在向基于 DevOps 的方式演进，进而支持持续开发与持续部署流水线，并逐渐采用基于容器及其编排机制的云原生架构方案。

1.2.2 国内研究与产品现状

国内云存储领域的发展主要由阿里云 OSS、腾讯云 COS 及华为云 OBS 等大型云服务商的对象存储服务支撑，企业网盘、内容分发及多媒体平台等场景对其依赖度较高。在学术研究层面，近三年的中文文献主要集中于三条主线^{[8][9]}：一是为 Docker 与 Kubernetes 容器化及微服务架构在中小型 Web 系统工程实践中的应用；其二为 Office 文档、图片及视频等资源的预览转码优化与缓存策略研究；其三为面向 SaaS、多租户及教学平台的轻量级权限隔离与数据归属设计。此外，多篇高校学位论文亦对 Spring Boot 与 Vue 前后端分离技术栈下的云文件系统进行了比较完整的描述^{[10][11][12]}，构成了本课题工程实现层面比较直接的参考。

1.2.3 现存问题与本课题切入点

结合上述国内外研究与工程实践能看到，云文件管理类系统在功能完整性与技术储备方面已经比较成熟，但在以下三个方面与本科毕业设计场景仍存在一定差距。第一，多数研究以中大型生产环境为研究对象，主要关注分布式部署与高可用性问题，对答辩场景要求的“环境快速复现”与“单机演示稳定”等诉求尚缺乏专门讨论。第二，预览与分享链路在反向代理叠加签名校验的组合使用情境下，易出现路径错乱与防盗链失效等问题，公开材料中对完整修复过程的描述比较有限。第三，毕业设计场景中常见的双账号演示需求，目前尚未形成相对统一的轻量级权限隔离方案，部分研究直接采用完整 RBAC 模型，其工程开销与课题规模之间存在比较明显的不匹配^[13]。

本课题正是以上述三类问题为切入点，在已有云文件管理系统工程范式的基础上，将研究主线收敛至容器化部署、预览访问链路修复及轻量级双用户隔离三个方向，并给出与真实代码相对应的实现方案，进而通过部署验证与功能测试构成完整的研究闭环。

1.3 研究内容

本文的研究内容主要落在系统设计、关键实现、运行验证三块上，具体包括下面五项工作。

其一，明确系统边界与功能闭环。本系统被定位为一套轻量级演示系统，仅保留 `admin` 与 `demo` 两类账号，文件归属依赖 `lfs_file.user_id` 字段予以标识，并以登录、上传、目录、预览、分享、回收站及存储统计构成完整的业务链路。

其二，完成系统的总体设计。采用前后端分离的 B/S 架构，分别针对表现层、业务层、数据与资源层及部署层四个层次给出模块划分、数据库关系与网络访问关系，并预留总体架构、网络访问、功能模块、E-R 与接口关系五张关键图。

第三件事是做轻量级双用户隔离机制。

在 `CurrentUserService` 和 `SchemaCompatibilityRunner` 的基础上，按当前用户身份在数据库查询层动态拼归属条件。同时在系统启动时自动把字段、索引、演示账号补齐，兼容旧的 MySQL 数据卷。

第四件事是修复并统一文件预览访问链路。在 `FilePreviewController` 里收敛 `/api/files`、`/api/thum`、`/api/trans` 这三类前缀的路径归一化处理，再配合 `secret` 和 `expire` 做签名校验，把原文件预览、缩略图预览、转码资源访问纳入统一控制。

最后一件是容器化部署和运行验证。系统用 `Docker Compose` 编排 `lfs-nginx`、`lfs-backend`、`lfs-mysql`、`lfs-redis` 四个核心容器，本地目录挂载做持久化。我用功能测试、单元测试、异常场景测试三种方式交叉验证，覆盖了容器启动、登录认证、文件管理闭环、预览链路修复、分享访问、双用户隔离这几个关键场景。

1.4 论文组织结构

全文共分五章。

第 1 章为绪论，介绍研究背景、国内外研究现状、研究内容以及论文组织结构。

第 2 章对系统实现所使用的基础知识与相关技术进行介绍，覆盖前端、后端、数据存储、容器化部署及文件预览相关技术栈，并对各类技术给出在本课题中的选型理由。

第 3 章为概要设计部分，阐述系统总体架构、功能模块划分、数据库结构及网络访问关系，并给出系统总体架构图、网络访问关系图、功能模块结构图、数据库 E-R 图及接口关系图。

第 4 章为详细设计与系统实现部分，按模块对用户认证、文件管理与双用户隔离、预览与分享、回收站与存储统计、容器化部署等内容予以展开，并结合关键代码、流程图与系统截图阐明实现细节。

第 5 章为测试和验证，给出测试方案、测试数据、功能测试结果、异常与边界测试结果以及结果分析。

最后给出全文结论，对研究工作进行总结，并讨论当前实现的不足以及未来可拓展的方向。

第 2 章 基础知识与相关技术

本系统所涉及的技术可划分为前端、后端、数据存储、容器化部署与文件处理五个方面，以下按其在系统中的使用顺序依次展开。前端采用 Vue 3、Element Plus、Vite 与 Vue Router；后端采用 Spring Boot、MyBatis-Plus、JWT 与 BCrypt；数据存储由 MySQL 8 与 Redis 7 协同承担；部署层整合 Nginx、Docker 与 Docker Compose；文件处理则依托 Aspose、PDFBox 与 JAVE 等工具库实现多格式预览。各类技术均结合课题需求说明其选型理由，最后单独设一小节，阐述本地文件访问所采用的签名校验机制。

2.1 前端技术

2.1.1 Vue 3

前后端分离架构是当前 Web 应用的主流开发模式之一：前端通过 JavaScript 框架构建用户界面并以 Ajax 方式调用后端接口，后端以 RESTful API 形式提供数据与业务逻辑服务，两者通过 JSON 等统一数据格式完成通信，从而有利于前后端并行开发、独立部署与多终端复用。本系统前端基于 Vue 3 构建。Vue 是一套用于构建用户界面的渐进式 JavaScript 框架，其核心库仅关注视图层，易于上手并便于与第三方库或既有项目整合。Vue 3 在内部采用基于 Proxy 的响应式系统，可更有效地追踪数据变化，并通过组合式 API 将响应式数据声明、计算属性、副作用与生命周期钩子集中于 setup 函数之中；相较于 Vue 2 的选项式 API，其在可读性与逻辑复用性方面具有较明显的优势。在本系统中，文件列表、目录树、登录页、预览弹窗、分享页及回收站等模块被划分为相互独立的组件，组合式 API 用于实现视图状态与业务逻辑的整合，并通过状态注入与事件机制实现组件间联动，从而在表单、列表与弹窗等交互密集的场景下保持稳定的运行表现。

在代码组织方式上，本系统按页面进行模块划分，登录、首页、文件管理、分享访问、转码进度及回收站等各模块相互独立、耦合程度较低，便于后续在截图补充、样式调整及接口扩展等环节中将改动范围控制在较小区间。

2.1.2 Element Plus

桌面端控件库选用 Element Plus，该库为基于 Vue 3 的桌面端组件库，所覆盖的组件类型包含表格、表单、对话框、分页、消息提示及上传等绝大多数后台管理类系统所需要的控件。在本系统中，文件列表的虚拟滚动、删除操作的二次确认、上传进度条、分享信息面板及回收站还原按钮等功能，均在该组件库的原生组件基础上经过适当配置后接入。该方式不

仅降低了自行实现等同水准控件所需的工作量，同时也保证了整体页面在视觉风格上的一致性。

2.1.3 Vite 与 Vue Router

构建工具选用 Vite。其底层基于 ESBUILD，并结合原生 ESM 的按需加载机制，使本地启动与热替换效率较传统打包器有显著提升；对于毕业设计中需反复调整样式与交互的场景，该特性较为契合。前端路由采用 Vue Router，将登录、主界面、分享访问、转码进度与回收站等一级页面进行注册，构成前后端分离架构下的统一入口分发机制。

结合上述前端选型可以说，Vue 3、Element Plus、Vite 及 Vue Router 所构成的技术组合可以基本覆盖本课题所涉及的全部交互需求：Vue 3 支持按业务领域进行组件划分；Element Plus 提供了文件管理类系统所需的常用控件；Vite 显著降低了开发期的等待时间；Vue Router 负责前端路由调度。在文件列表、目录树及预览弹窗等多视图、强交互的场景中，该组合并未表现出明显的功能短板。

2.2 后端技术

2.2.1 Spring Boot

Spring Boot 是基于 Spring 框架的快速开发脚手架，通过约定优于配置、起步依赖与自动配置等机制，显著简化了 Spring 应用的初始化与构建过程，使开发者只需关注业务逻辑即可快速搭建独立运行、生产就绪的 Web 应用。本系统后端采用 Spring Boot 3.2.2，运行于 Java 17 环境。选型理由有三：其一，约定优于配置的机制简化了应用的搭建与配置，便于将精力集中于业务逻辑实现；其二，JDK 17 为长期支持版本，在性能与稳定性方面较为可靠；其三，内嵌 Tomcat 与 Starter 依赖管理机制降低了工程配置的复杂度。

2.2.2 MyBatis-Plus

持久化层选用 MyBatis-Plus 3.5.5，作为 MyBatis 的增强工具，在保留原有 MyBatis 功能特性的基础上，提供了通用 Mapper、通用 Service 以及条件构造器 Wrapper 等扩展能力，可在少量配置的前提下完成单表层面的大部分 CRUD 操作，与本系统以结构化数据为主的文件管理场景较为契合。

在持久化方案的选择上，相较于全自动的 JPA/Hibernate，MyBatis-Plus 在保留 SQL 可控性的同时减少了样板代码，更便于对文件元数据查询进行针对性优化；而相较于原生 MyBatis 需手写大量 XML 映射，其通用 Mapper 与条件构造器又显著降低了单表 CRUD 的开发成本。对于以结构化数据、单表操作为主的文件管理场景，该方案在开发效率与 SQL 可控性之间取得了较好的平衡。

2.2.3 安全机制与支撑库

登录态用 JSON Web Token (JWT) 来传, 密码落库走 BCrypt 哈希加自动随机盐。JWT 是一种开放标准 (RFC 7519), 把用户标识、角色、过期时间打包成一个带数字签名的 token, 服务端不用再维护会话状态。BCrypt 基于 Blowfish 密码算法, 通过提高 cost 因子来延长离线穷举的时间, 再用盐值来挡彩虹表攻击。这两套机制相互配合, 无需引入完整的 RBAC 即可覆盖登录认证、双账号识别与接口准入等基本需求。

除上述核心框架外, 项目还引入了若干辅助库。数据库连接池采用 Druid, 并借助其提供 SQL 监控与防护能力。JSON 的序列化与反序列化由 Fastjson2 完成, 在处理大对象时具有较稳定的表现。Lombok 用于消除实体类、DTO 与工具类中重复的 getter/setter 代码; MapStruct 用于在领域实体、DTO 与 VO 之间进行对象映射, 相较手写代码或 BeanCopier 更为清晰明确。上述库虽不直接承担业务逻辑, 但对代码可读性与运行稳定性具有不可忽视的支撑作用。

2.3 数据存储技术

2.3.1 MySQL

MySQL 是最流行的开源关系型数据库管理系统之一, 因其体积小、速度快、总体拥有成本低且开放源码, 被广泛应用于各类 Web 系统的数据存储^[14]。本系统底层关系型数据库选用 MySQL 8, 用于存储用户信息、文件元数据、目录关系、分享记录、回收站与转码任务等数据。其中, 文件表 lfs_file 在原有字段基础上新增了用于标识归属的 user_id 列, 并在该列上建立辅助索引; 该列是双账号隔离机制正常运作的基础, 统计、分享与回收站等模块在数据过滤过程中亦依赖该列。MySQL 支持事务、外键与行级锁等特性, 遵循 ACID 原则, 能够满足结构化数据在可靠性方面的要求。

2.3.2 Redis

Redis 是一个开源的、基于内存并支持持久化的高性能键值型数据库, 其值可为字符串、哈希、列表、集合与有序集合等多种数据结构, 常被用作缓存、会话存储与消息中间件^[15]。本系统将生命周期较短、读写较为频繁的数据存储于 Redis 7 之中, 主要包括分片上传的中间状态、上传进度、转码任务的进度推送、部分会话信息及偶发的热点查询缓存。

Redis 作为一种基于内存的高性能键值型数据库, 支持字符串、哈希、列表、集合及有序集合等多种数据结构, 常被用作缓存、会话存储及消息中间件。本系统将生命周期较短、读写较为频繁的数据存储于 Redis 7 之中, 主要包括分片上传过程中的中间状态、上传进度、转码任务的进度推送、部分会话信息及偶发的热点查询缓存结果。将该类临时数据从 MySQL 中分离的设计, 一方面有助于减轻数据库压力, 另一方面在上传与转码等高并发场景下可避免因频繁写库而引发性能下降。

将此类临时数据从 MySQL 中分离的设计具有两方面意义：一方面，关系型数据库借助事务、索引与外键承担文件归属、权限关联等结构化查询需求；另一方面，Redis 将上传分片、转码进度等短生命周期状态置于内存中处理。由此，业务数据与运行时状态在存储介质层面相互分离，既减轻了数据库压力，也避免了在上传与转码等高并发场景下因频繁写库而引发的性能下降。

2.4 部署技术

2.4.1 Nginx

Nginx 是一款轻量级的 Web 服务器与反向代理服务器，由 Igor Sysoev 开发，具有内存占用低、并发能力强的特点，被百度、京东、淘宝等大量站点采用。在本系统部署中，外部进入系统的全部流量首先到达 Nginx，由其同时承担两项职责：一是直接对外提供前端构建产物（HTML/JS/CSS 等静态资源），二是将 /api 前缀的请求反向代理至 Spring Boot。预览链路在此基础上需作进一步处理——/api/files、/api/thum、/api/trans 三类请求需按统一规则转发给后端，并将 Host、X-Real-IP 等头部原样透传，否则后端的签名校验将无法获取正确的请求上下文。

2.4.2 Docker 与 Docker Compose

Docker 是当前主流的开源应用容器引擎，其基于 Linux 内核的 Namespace 与 Cgroups 机制，实现进程、网络与文件系统等资源的轻量级隔离^[4]；镜像由多层文件系统叠加而成，并通过联合挂载在只读层之上增加可写层。相较于传统虚拟机，容器具有启动速度快、资源占用少、易于移植与版本化管理等优势，已成为构建云原生应用的事实标准。Docker Compose 是用于定义和运行多容器应用的工具，通过 YAML 文件配置应用所需的全部服务，并以单条命令完成统一创建与启动。

本系统整体由 lfs-nginx、lfs-backend、lfs-mysql、lfs-redis 四个容器构成，由 Docker Compose 统一编排。镜像层面固定 JDK、Node、Nginx 及中间件版本，以避免不同机器上的行为差异；运行层面通过 volume 将上传目录、MySQL 数据目录、Redis 持久化目录与 Nginx 配置目录映射至宿主机，从而在容器销毁重建后业务数据不致丢失。整套环境仅需一条 docker compose up -d --build 命令即可从零启动，在更换机器或演示场地时显著降低了环境配置成本。

在编排方案的选择上，本系统未采用 Kubernetes，而选用 Docker Compose。其原因在于：Kubernetes 面向大规模、高可用的集群编排，引入了较高的部署与运维复杂度；而本课题为面向本科毕业设计的单机演示系统，服务数量有限，无弹性伸缩与多节点调度需求。Docker Compose 以单文件描述多服务的依赖、版本与网络配置，使开发、测试与答辩演示三套环境共用同一份描述文件，更契合演示场景对部署可重复性与轻量化的要求。

2.5 文件处理与预览技术

Office 文档是企业办公中最常见的电子文档形式，其中 Word（.doc/.docx）用于文字处理，Excel（.xls/.xlsx）用于电子表格处理，PowerPoint（.ppt/.pptx）用于演示文稿制作，三者构成了 Microsoft Office 办公套件的核心组成部分。在线预览技术通过将上述格式文档转换为浏览器原生支持的格式（如 PDF、HTML 或图片），实现无需安装客户端即可查看文档内容的目标。文件管理系统真正麻烦的地方，并不在于“是否能上传”，而在于“是否能在浏览器里稳定预览”。系统需要支持图片、Office 文档、PDF、音频以及视频等多种文件类型，并在统一的访问链路下提供原文件与缩略图两种预览方式。

对于 Word、Excel、PowerPoint 三类文档，系统采用 Aspose Words、Aspose Cells、Aspose Slides 在服务端进行统一转换。原始 Office 文档直接在浏览器中打开往往存在兼容性问题或需安装插件，因此系统在文件落地后立即将其转换为 PDF 或 HTML 等通用格式，前端仅需通过 iframe 或预览组件加载，无需依赖用户本地的 Office 版本。

2.5.1 文档转换与 PDF 渲染

PDF（Portable Document Format，便携式文档格式）是由 Adobe 公司于 1993 年推出的一种文件格式，用于以与应用程序、操作系统、硬件无关的方式呈现文档。每个 PDF 文件包含固定布局的平面文档的完整描述，包括文本、字体、矢量图形、光栅图像和其他显示所需要的信息。PDF 文件的渲染与封面提取由 PDFBox 负责处理。在本系统的预览链路中，PDF 既作为直接被预览的目标格式，也作为 Office 文档转换的中间产物——所有“预览”类需求均先归一化为 PDF 格式，再统一进入渲染流程，使整条链路保持单一主干，便于故障定位，亦便于在新增格式时仅需于前端增加一道转换。

2.5.2 音视频处理

HTML5 video 是 HTML5 标准中引入的视频播放元素，用于在网页中嵌入视频内容，不再依赖第三方插件，支持 MP4、WebM、Ogg 等多种格式，并自带播放、暂停、音量调节与全屏切换等基本控制接口。音视频文件与普通文档差异较大——体积大、编码格式繁杂、客户端兼容性较差，因此系统为其单独设计了一条处理流程：视频转码与封面提取由 JAVE 库完成，前端再通过 HTML5 video 或专用播放器组件加载转码后的资源。先在服务端完成格式归一化、再由浏览器加载的方式，可有效避免因特殊编码而导致的画面异常或无法播放问题。

2.5.3 本地文件访问与签名防盗链

上传完成的文件最终被持久化存储于宿主机的指定目录之中，外部访问需经由 Nginx 反向代理予以转发。若不对该访问链路加以约束，外部用户可直接通过路径拼接获取文件资源，

因此须在中间环节引入签名校验机制以阻止盗链与未授权访问。本系统采用 `secret + expire` 双参数模式，其整体逻辑如下：

（1）签名生成。当用户发起文件预览请求时，后端依据访问路径、向后偏移的过期时间戳与服务端密钥，按 `secret = HMAC(server_key, path + "|" + expire)` 计算签名，再将 `secret` 与 `expire` 拼接至预览 URL 之后。其中 `expire` 采用 Unix 秒级时间戳，通常设定为生成时刻之后的数分钟。

（2）签名校验。当 `/api/files`、`/api/thum`、`/api/trans` 三类资源被请求时，后端依次执行三步校验：首先判断 `expire` 是否早于当前时间，若已过期则予以驳回；随后采用同一密钥与算法重新计算签名，并与请求传入的 `secret` 进行常量时间比较，若不一致则驳回；最后对解析出的真实路径进行越界检查，以阻止 `../` 等路径穿越攻击。

（3）安全语义说明。密钥仅保存在后端，外部无法构造合法签名。由于 `expire` 字段被纳入签名计算范围，攻击者也无法在不破坏签名的前提下延长 URL 的有效期。该机制实现成本较低，但能够同时覆盖防盗链与时效控制两类需求。分享码访问、回收站访问与缩略图访问共用同一套校验逻辑。

文件处理与预览选型理由可以这么讲：`Aspose` 系列、`PDFBox` 及 `JAVE` 均为 Java 生态中已经比较成熟的开源或商业类库，无需额外部署转码服务即可实现多格式预览能力；`secret + expire` 签名机制则在未引入完整鉴权框架的前提下，将本地文件访问统一收敛于一道安全边界之内。

2.6 本章小结

本章按“从前端到后端”的顺序梳理了系统所依赖的全部关键技术。前端层由 `Vue 3`、`Element Plus`、`Vite` 与 `Vue Router` 共同支撑多视图前端；后端落在 `Spring Boot 3.2.2`、`MyBatis-Plus`、`JWT` 与 `BCrypt` 上承担业务接口和基础安全；数据层由 `MySQL 8` 负责持久化、`Redis 7` 负责临时状态，二者职责相互独立；部署层由 `Nginx`、`Docker` 与 `Docker Compose` 协同完成静态资源分发、反向代理与多服务编排；文件处理层则借助 `Aspose`、`PDFBox` 与 `JAVE` 将多格式预览统一收敛至同一条处理链路。针对本地文件访问场景，又额外引入了一道 `secret + expire` 签名机制，为后续设计与实现章节中的预览链路、分享访问与缩略图访问提供了统一的安全边界。

第 3 章 概要设计

3.1 需求分析

在系统设计与实现之前，需结合本科毕业设计的应用场景对系统需求进行梳理。本系统面向轻量级演示场景，使用者主要为系统管理员与演示用户两类角色，核心诉求是在可反复部署的前提下完成文件管理业务的完整闭环，并实现轻量级的用户数据隔离。基于这一定位，下文分别从功能需求与非功能需求两个方面对系统需求加以说明。

3.1.1 功能需求

依据系统的业务链路，可将功能需求划分为用户认证、文件管理与双用户隔离、预览与分享、回收站与存储统计、容器化部署五个方面，各功能模块的具体需求如表 1 所示。

表 1 系统功能需求

功能模块	功能需求描述
用户认证	支持 admin 与 demo 两类账号登录；登录后签发 JWT 令牌并在后续请求中解析当前用户；登录态失效时给出明确反馈
文件管理与双用户隔离	支持文件上传（含 MD5 秒传与分片上传）、列表浏览、目录树管理、移动、重命名与命名冲突处理；所有文件按归属用户进行数据隔离
预览与分享	支持 Office 文档、PDF、图片与音视频等多种格式的在线预览；支持以分享码加有效期的方式对文件进行匿名分享
回收站与存储统计	支持文件的软删除、还原、彻底删除与物理清理；支持按用户维度统计文件数量与空间占用
容器化部署	支持基于 Docker Compose 的一键部署；保证数据卷持久化；支持通过配置项与环境变量调整部署参数

3.1.2 非功能需求

除上述功能需求外，系统在性能、安全性、可移植性与可维护性等方面亦需满足相应要求。

在性能方面，系统通过 MD5 秒传避免相同内容文件的重复传输，借助分片上传应对较大文件，并利用 Redis 缓存上传上下文以降低数据库压力，从而保证演示过程中上传与预览操作的响应效率。

在安全性方面，系统采用 JWT 进行登录态管理，用户口令经 BCrypt 加密存储；文件预览访问通过基于 HMAC 的签名校验（secret 与 expire）实现防盗链；并通过 user_id 字段保证两类用户之间的数据相互隔离。

在可移植性方面，系统整体基于容器化交付，借助 Docker Compose 实现一键部署与数据持久化，可在具备 Docker 环境的不同主机上反复复现，满足答辩演示对部署可重复性的要求。

在可维护性方面，系统按业务链路进行模块化划分，各模块职责清晰；并通过启动期数据库兼容补丁（SchemaCompatibilityRunner）自动完成旧库结构的升级与归属字段回填，降低了版本迭代与环境迁移的维护成本。

3.2 设计目标与系统边界

本系统被定位为一套轻量级演示系统，主要面向本科毕业设计场景使用。对外仅保留两类账号——admin 与 demo，前者具备查看全量数据及执行管理操作的权限，后者则仅能在自身的文件空间内进行操作。文件的归属关系统一由 lfs_file.user_id 字段予以标识，分享访问路径则采用“分享码 + 有效期 + 匿名访问”这一较为轻量的方案^[6]。

在功能边界上，本课题不追求构建完整的多租户体系，也不引入 RBAC 模型，更不会做组织架构、审批流程、配额管理这些企业级特性。设计重心放在四条主线上：文件管理闭环、容器化部署、预览访问链路、轻量级双用户隔离。上述主线需在容器环境下稳定运行，并能够被反复演示与复现。具体的设计目标可归纳如下：一是借助 Docker Compose 实现整体系统的一键启动，并通过本地目录持久化保证数据可重复使用；二是登录、上传、目录、列表、预览、分享、回收与统计这一业务链需保持完整且可演示；三是预览访问路径 /api/files、/api/thum、/api/trans 在反向代理环境下能稳定工作。

具体的设计目标可归纳为以下五点。一是借助 Docker Compose 实现整体系统的一键启动，并通过本地目录持久化保证数据的可重复使用；二是登录、上传、目录、列表、预览、分享、回收与统计这一业务链需保持完整且可演示；三是预览访问路径 /api/files、/api/thum、/api/trans 这三类需共用同一套签名校验；四是 admin 与 demo 之间的数据可见范围依据 user_id 严格隔离；五是即便接入旧版 MySQL 数据卷，启动时缺失的字段与索引也应能够自动补齐，从而避免历史数据丢失。

3.3 总体架构设计

3.3.1 层次结构

整套系统按前后端分离的 B/S 形态搭，从上往下被切成四层：表现层、业务层、数据与资源层、部署与代理层。最上面一层是 Vue 3 + Element Plus + Vite + Vue Router 一起撑起来

的多视图前端，主要负责把页面渲染出来并接住用户操作；往下一层是 Spring Boot 3.2.2 跑起来的业务接口，登录、文件管理、目录管理、分享、回收站、预览、存储统计都集中在这里；再往下数据和资源那一层各司其职——MySQL 8 把元数据落进表里，Redis 7 接住那些短生命周期的状态，物理文件则直接落到宿主机本地目录里；最底下的部署层把 Nginx 摆在前面做统一入口，再由 Docker Compose 把 lfs-nginx、lfs-backend、lfs-mysql、lfs-redis 这四个容器编到一起协同跑^[11]。

系统总体架构如图 3.1 系统总体架构所示。

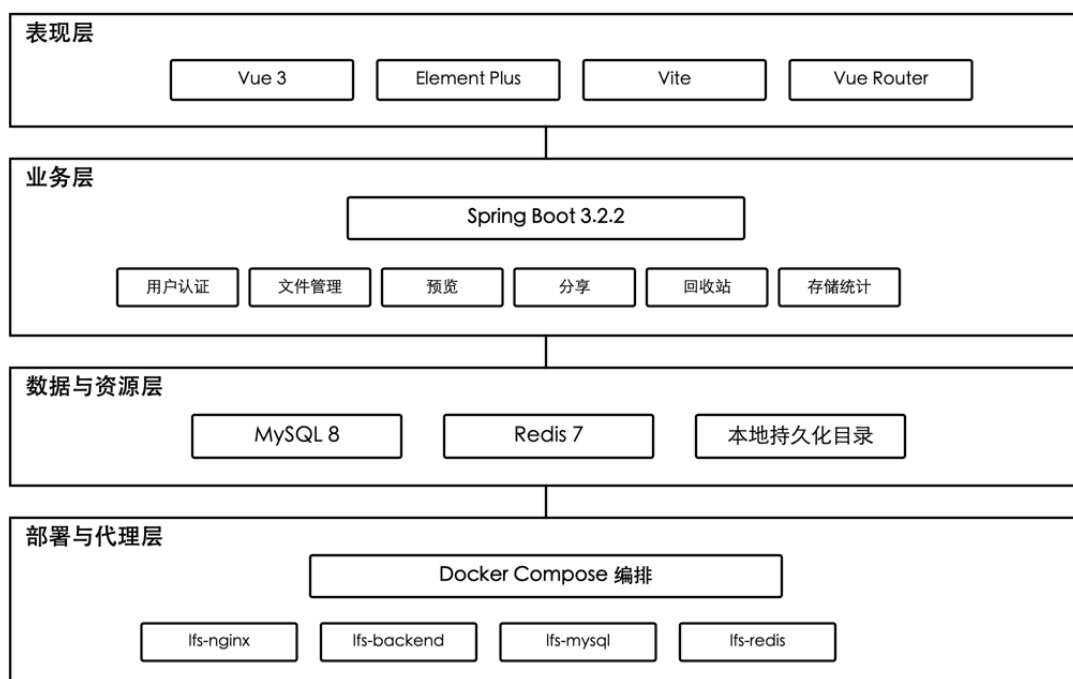


图 3.1 系统总体架构

3.3.2 网络访问关系

从访问链路上看，浏览器发出的请求首先到达 Nginx；Nginx 再按路径前缀进行分流：以 /api 开头的请求转发给后端 Spring Boot，其余请求则按静态资源直接返回前端构建产物。在反向代理转发过程中，Nginx 同步将 Host、X-Real-IP 与原始 URI 等头部一并传递给后端，为后端的签名校验和日志审计提供必要依据。文件预览、缩略图与转码资源走的是另一条分支：客户端访问 /api/files、/api/thum、/api/trans 时，Nginx 仍先将请求转发回后端，后端在完成 secret 与 expire 签名校验之后，再从宿主机持久化目录中读取真实文件并返回。整体这一套访问关系呈现在图 3.2 网络访问关系上。

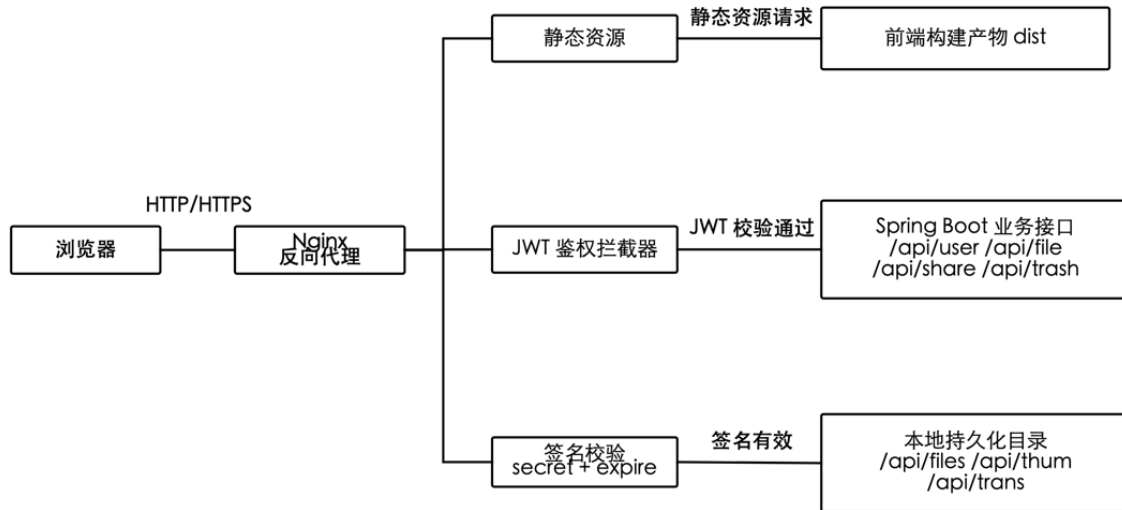


图 3.2 网络访问关系

3.4 功能模块划分

3.4.1 模块组织

本系统按业务链路对功能模块进行组织，模块之间通过 REST 接口、当前用户上下文及共享数据库三种方式协同工作。具体模块划分如下：用户认证模块负责登录与当前用户识别；文件管理模块覆盖上传、列表、目录、移动及删除等核心操作；预览模块统一处理原文件预览、缩略图预览及转码文件访问；分享模块负责分享码生成、有效期校验及匿名访问；回收站模块记录删除、恢复及彻底删除等操作轨迹；存储统计模块按用户隔离汇总空间使用情况；兼容补丁模块则在系统启动期对旧库字段进行检测与回填^[16]。

各模块的关键依赖如表 2 所示。

表 2 系统功能模块及其关键依赖

模块	主要职责	关键依赖（数据表/类）
用户认证模块	登录鉴权、token 解析、当前用户识别	sys_user 表 / JWT 工具类 /CurrentUserService
文件管理模块	上传、列表、目录、重命名、移动、删除	lfs_file 表 /FileMapper / MyBatis-Plus
预览模块	原文件预览、缩略图预览、转码文件访问	FilePreviewController/ secret + expire / 本地资源目录
分享模块	分享码生成、有效期校验、匿名访问	lfs_file_share 表 /FileShareServiceImpl
回收站模块	删除、恢复、彻底删除、清空	lfs_file_trash 与 lfs_file_trash_detail 表
存储统计模块	用户隔离的空间使用统计	FileMapper.sumTotalFileSizeByUserId/lfs_file.user_id
兼容补丁模块	启动期字段、索引与演示账号自动补齐	SchemaCompatibilityRunner/ JDBC Template

各模块之间的协作关系如图 3.3 所示。

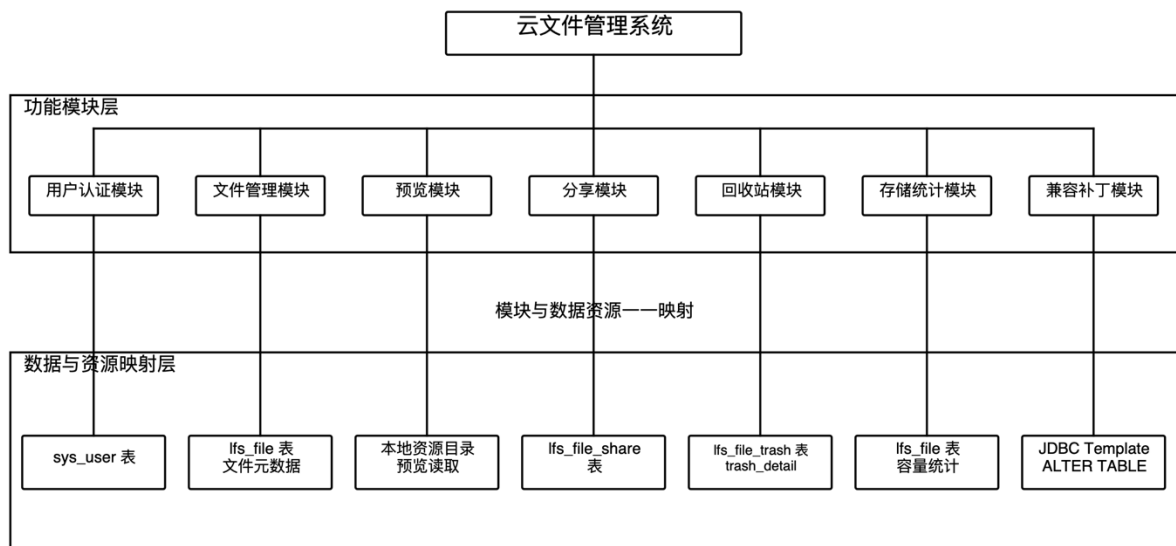


图 3.3 系统功能模块结构图

3.4.2 模块协作时序

业务执行的总体协作顺序如下：用户在访问受保护接口之前，认证模块先从请求头中解析 JWT，获取当前用户 ID 并存入上下文；文件管理模块在执行 SQL 查询时，由 `CurrentUserService` 提供当前用户 ID，并作为过滤条件追加至 `WHERE` 子句中；预览与分享模块在生成访问 URL 时，由签名工具计算 `secret` 与 `expire`；客户端使用该 URL 访问资源时，预览模块负责签名校验与文件返回；删除操作经回收站模块流转至 `lfs_file_trash`，回收站操作不直接修改 `lfs_file` 主表中的物理记录；统计模块则在用户进入主界面与回收站界面时被触发，按 `user_id` 汇总实时数据。该协作机制结构比较简洁、边界亦比较清晰，符合本课题作为轻量级演示系统的定位。

3.5 数据库设计

3.5.1 核心表结构

数据库以 `sys_user` 和 `lfs_file` 这两张主表为核心。`sys_user` 存账号信息、加密密码、角色标识和基础信息；`lfs_file` 存文件和目录的核心元数据，里面有父目录 ID、归属用户 ID（`user_id`）、文件名、扩展名、大小、MIME 类型、状态字段、回收站标记。围绕这两张主表，系统还建了 `lfs_file_share` 存分享码和有效期、`lfs_file_trash` 加 `lfs_file_trash_detail` 存回收站主从关系、`lfs_trans_file` 加 `lfs_trans_progress` 存转码任务和进度、`lfs_file_thum` 存缩略图引用、`lfs_trans_template` 存转码模板这几张扩展表^[17]。

3.5.2 双用户隔离的核心字段

`lfs_file.user_id` 是本系统双用户隔离的核心字段。这个字段决定了每条文件记录归谁所有，也就决定了 `demo` 账号在文件列表、预览、分享、回收站这些业务里能看到的数据范围。我在 `lfs_file.user_id` 上建了辅助索引 `idx_user_id`，保证按用户过滤查询的执行效率。所有按用户隔离的统计与查询语句（比如 `sumTotalFileSizeByUserId`、`countByUserId`）都把这个字段当成唯一过滤维度，避免数据交叉。

系统启动期会由 `SchemaCompatibilityRunner` 对该字段做兼容性检查。如果旧版 MySQL 数据卷里的 `lfs_file` 表没有 `user_id` 字段，自动执行 `ALTER TABLE` 把列加上并设默认值；如果缺少 `idx_user_id` 索引，自动建一个；如果 `demo` 演示账号缺失，也会自动补种。这套机制保证旧库升级之后还能正常运行，省去人工介入的步骤。

系统数据库实体关系如图 3.4 所示。图中以主键、外键以及 `user_id` 关联线突出标注核心字段及其引用方向：`sys_user.id` 作为主键被 `lfs_file.user_id` 引用，`lfs_file.id` 作为主键被 `lfs_file_share.file_id`、`lfs_file_trash_detail.file_id` 与 `lfs_file_thum.file_id` 引用，从而形成以 `lfs_file` 为中心的星型结构。

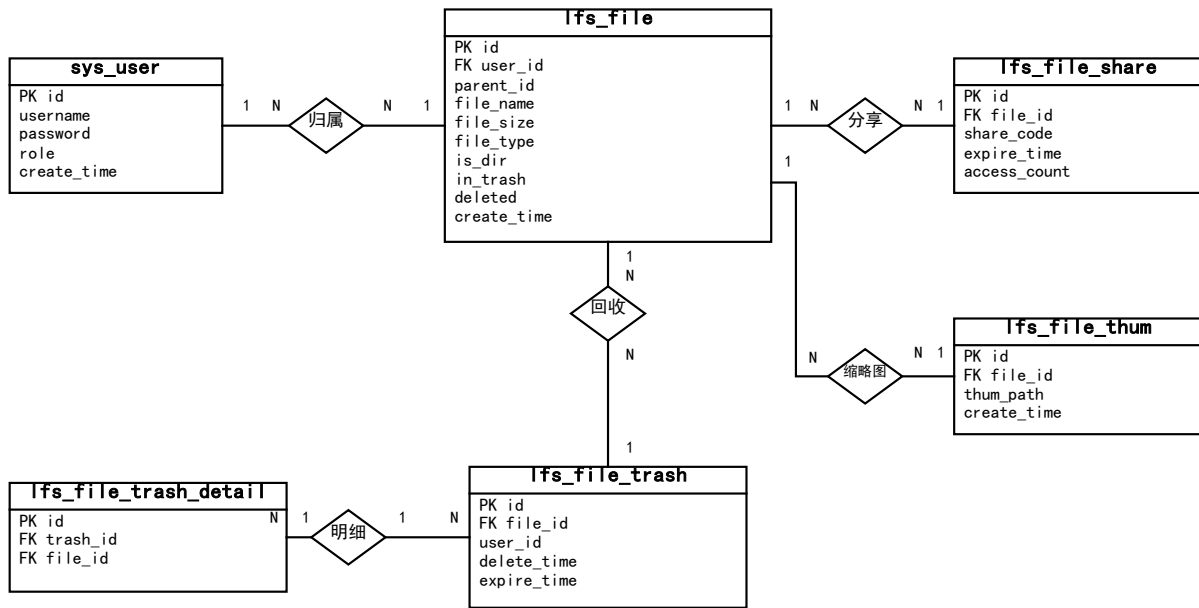


图 3.4 数据库表关系图

3.6 接口设计

3.6.1 REST 接口分组

REST 架构风格由 Roy Fielding 在其 2000 年的博士论文中正式提出，其核心约束包括客户端-服务器架构、无状态通信、可缓存性、统一接口、分层系统以及按需代码六项^[18]。本系统按 REST 风格组织对外接口，所有资源均通过 URI 标识，通过 HTTP 方法（GET、POST、PUT、DELETE）表达对资源的具体操作。系统对外接口采用 REST 风格组织，按业务域划分为四组：/api/user/ 负责登录与用户信息查询，/api/file/ 负责文件与目录的增删改查，/api/share/ 负责分享码生成与匿名访问，/api/trash/ 负责回收站操作。所有需要登录态的接口都通过 JWT 拦截器进行 token 校验；分享与预览接口则采用签名校验机制对外开放。

3.6.2 预览访问与普通路由的分离

为了避免反向代理路径错乱以及防盗链失效，系统对文件预览访问与普通业务路由进行了明确的分离。普通业务路由（/api/user、/api/file、/api/share、/api/trash）由 Nginx 按统一规则反向代理到后端，并经 JWT 校验之后进入业务逻辑；文件预览访问（/api/files、/api/thum、/api/trans）则由 Nginx 转发后，由后端 FilePreviewController 统一执行三步处理：先做路径

归一化（去除 /api 前缀、规范化资源路径），再做 secret 与 expire 签名校验，最后才是真实文件读取与流式返回。两类路径在 Nginx 配置层就已经被完全区分开了，从而避免普通业务路由被签名校验逻辑误拦截，也避免预览资源访问绕过签名进入业务接口^[10]。

系统接口与访问关系如图 3.5 所示，图中区分了带 JWT 认证的业务接口、带签名校验的预览接口以及匿名访问的分享接口这三类路径。

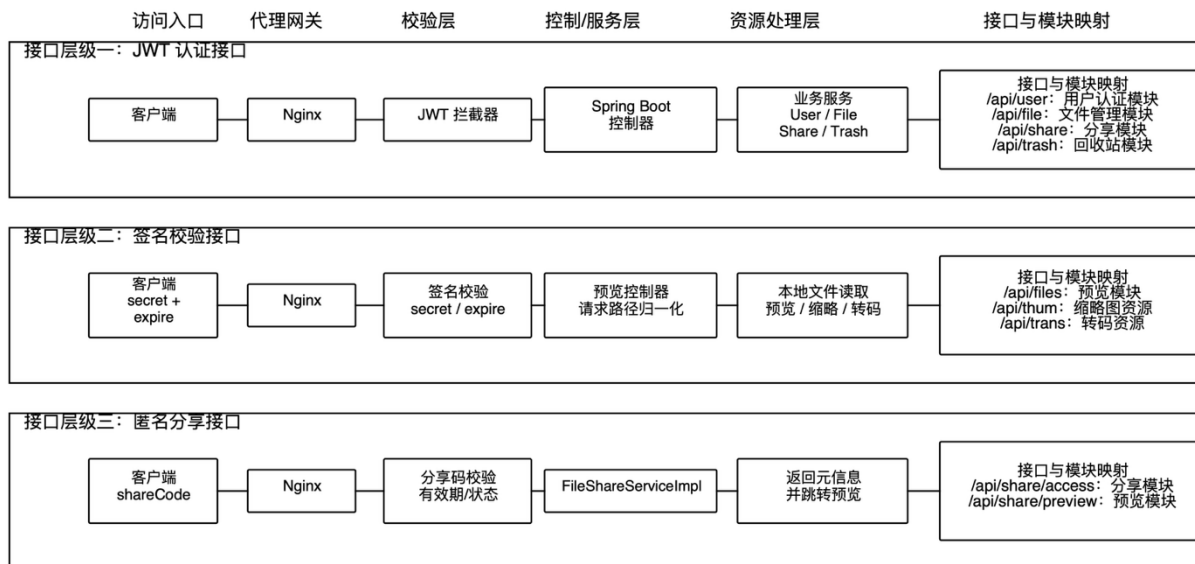


图 3.5 系统接口分层访问流程图

3.7 本章小结

本章首先对系统的功能需求与非功能需求进行了分析，并在此基础上按系统边界、总体架构、功能模块、数据库结构与接口设计五个方面完成了概要设计。系统整体定位为面向本科毕业设计场景的轻量级演示系统，在功能完整性与实现复杂度之间取得平衡，为后续详细设计与实现提供了清晰的结构依据。

第 4 章 详细设计与系统实现

本章承接第 3 章的概要设计，按“用户认证 → 文件管理与双用户隔离 → 预览与分享 → 回收站与存储统计 → 容器化部署”的顺序展开详细说明。每一节均包含模块职责、接口划分、关键流程、核心代码、文字说明及对应截图位；预览链路修复、双用户隔离及启动期兼容补丁等改造点将被单独提取并予以重点阐述。

4.1 用户认证模块实现

4.1.1 模块职责与接口划分

认证模块在系统中承担前置服务的职能——登录、令牌生成及当前用户识别均由该模块负责，其它业务模块在运行之前均需先经过该模块的处理。文件管理、回收站、分享及存储统计等模块在每次执行时均需获取“当前用户”信息，该步骤统一由认证模块予以完成。账号设置上仅保留两个：`admin` 与 `demo`，分别对应管理员视角与演示用户视角。前端登录页通过 `POST` 方式将用户名与密码提交至后端，由 `UserService` 完成校验；校验通过后，由 `JwtConfig` 签发 `token` 并返回前端，同时在响应中附加 `isAdmin` 标识，前端据此判断是否需要显示管理员菜单。

模块对外提供的核心接口如表 3 所示。

表 3 用户认证模块主要接口

接口路径	方法	主要用途	登录要求
<code>/api/user/login</code>	<code>POST</code>	账号密码登录，返回 <code>token</code> 与 <code>isAdmin</code> 标记	否
<code>/api/user/info</code>	<code>GET</code>	获取当前登录用户信息	是
<code>/api/user/logout</code>	<code>POST</code>	注销登录态	是

4.1.2 登录流程

登录流程整体走法呈现在图 4.1 用户登录流程图上。

登陆界面如图 4.2 所示。

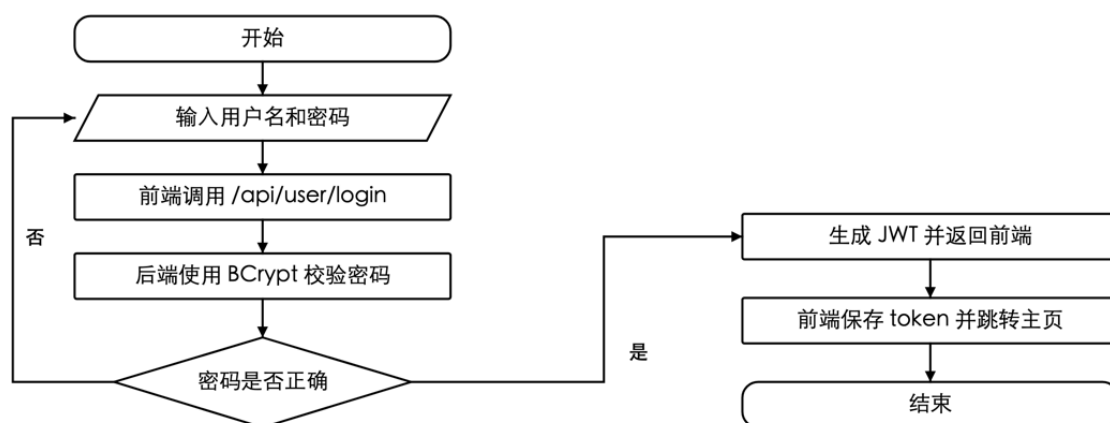


图 4.1 用户登录流程图

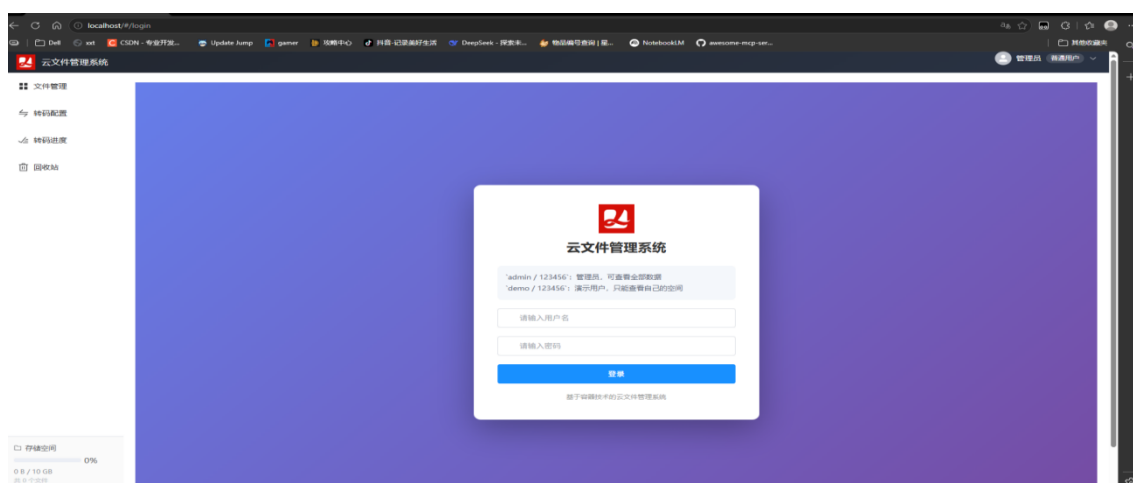


图 4.2 登陆界面

浏览器把用户名跟密码 POST 过来，后端先用 `UserService.findByUsername` 查一下账号到底存不存在，再用 `BCryptPasswordEncoder.matches` 把传过来的密码与表里那串哈希拿过来比一比。两步都过得去，就交给 `JwtConfig.generateToken` 把令牌拼出来——令牌里塞了用户编号、用户名以及签发时间，过期时间则由 `application.yml` 里的 `jwt.expire` 控一下。前端拿到 token 之后扔到 `localStorage`，后续每次请求由 `Axios` 请求拦截器顺手把 `Authorization` 头补上，到了后端入口由 `GlobalInterceptor` 来兜底校验。

4.1.3 当前用户解析服务

为了不让各业务模块重复解析 token，系统抽出了 `CurrentUserService` 这个统一入口，专门负责读取当前用户标识。其他业务模块只用调两个方法就能拿到当前用户编号和管理员身份。代码实现如下：

```
public Long getCurrentUserId() {
    String token = requireValidToken();
    Long userId = jwtConfig.getUserIdFromToken(token);
    Assert.notNull(userId, ErrorCode.TOKEN_INVALID);
    return userId;
}

public boolean isAdmin() {
    return UserService.ADMIN_USERNAME.equalsIgnoreCase(getCurrentUsername());
}
```

认证模块对外有两个核心方法：`getCurrentUserId` 和 `isAdmin`。所有受保护接口走 `requireValidToken` 拦截器，先把 token 的有效性查一遍。token 解析这一步封在 `JwtConfig` 里——签名校验、过期时间核对、用户标识提取都在里面，确保拿到的用户信息是合法令牌里来的。

这个服务被文件管理、分享、回收站、存储统计这些模块统一注入。它的上游是 JWT 拦截器和请求线程上下文，下游是各业务模块的 SQL 拼接逻辑——后者按 `getCurrentUserId` 和 `isAdmin` 的返回值决定是否在 WHERE 子句里加上 `user_id` 过滤条件。集中式的当前用户解析机制能避免不同模块独立解析 token 带来的实现差异，也方便后面扩展或者替换底层鉴权方案。

4.1.4 异常处理与登录态过期

为了在登录态失效时仍然能给出清晰反馈，认证模块在 `requireValidToken()` 中按“无 token / token 失效 / 解析异常”这三类情况分别抛出对应错误码，并由全局异常处理器 `GlobalExceptionHandler` 统一转换成 HTTP 401 状态码与可读消息。前端 `Axios` 拦截器在收到 401 之后会清空本地 token、跳转登录页并提示“登录已过期，请重新登录”，避免演示场景中出现接口报错但页面仍停留在主界面的不一致状态。前端路由守卫同时检查本地 token 是否存在，未登录访问受保护路由时直接跳转登录页，避免请求穿透到后端再被 401 拒绝。

4.2 文件管理与双用户隔离实现

文件管理模块负责目录创建、文件上传、列表展示、详情查看、重命名、移动及删除等核心操作，前端界面的绝大多数交互最终均落至该模块。相较于仅实现增删改查的常规文件管理模块，本系统在该模块中额外承担了轻量级双用户隔离任务——该任务为本课题三条研究主线之一。

4.2.1 上传与秒传实现

文件上传由前端 `Upload.vue` 组件、后端 `FileController.upload` 与 `FileService` 协同完成，整体可划分为 MD5 计算、秒传判定、分片上传、合并落盘与元数据写入五个阶段。用户选定文件后，前端首先计算文件的 MD5 值，并以该值向后端发起秒传查询；若后端检测到相同内容的文件已存在，则触发秒传，仅为当前用户写入一份归属记录，而不再重复传输文件内容；若不存在，则由后端分配 `uploadId`，并借助 `Redis` 记录本次上传的上下文与分片进度，最终由后端完成分片合并落盘，并向 `lfs_file` 表写入一条元数据记录。上传流程如图 4.3 所示，关键代码如下：

```
// FileService.uploadPrepare 关键片段
String md5 = request.getMd5();
File existing = fileMapper.findByMd5(md5);
if (existing != null) {
    fileService.quickUpload(existing, request, currentUserService.getCurrentUserId());
    return UploadResult.quick(existing.getId());
}
String uploadId = IdUtil.fastSimpleUUID();
redisTemplate.opsForValue().set(uploadKey(uploadId), request, Duration.ofHours(1));
return UploadResult.newUpload(uploadId);
```

该逻辑包含两条执行路径。当 `fileMapper.findByMd5` 返回已有记录时，表明系统中已存在相同内容的文件，此时 `quickUpload` 仅为当前用户复制一份元数据，不再占用额外的物理存储空间；当返回为空时，则以 UUID 作为 `uploadId`，将上传请求对象写入 `Redis` 并设置 1 小时有效期，供后续分片上传过程回查上下文。秒传机制在重复上传场景下可显著节省存储空间，相同内容的文件无论被多少用户上传，物理层面均仅保留一份。分片上下文存储于 `Redis` 而非数据库，旨在降低并发场景下的锁竞争，并提升临时状态的读写性能。

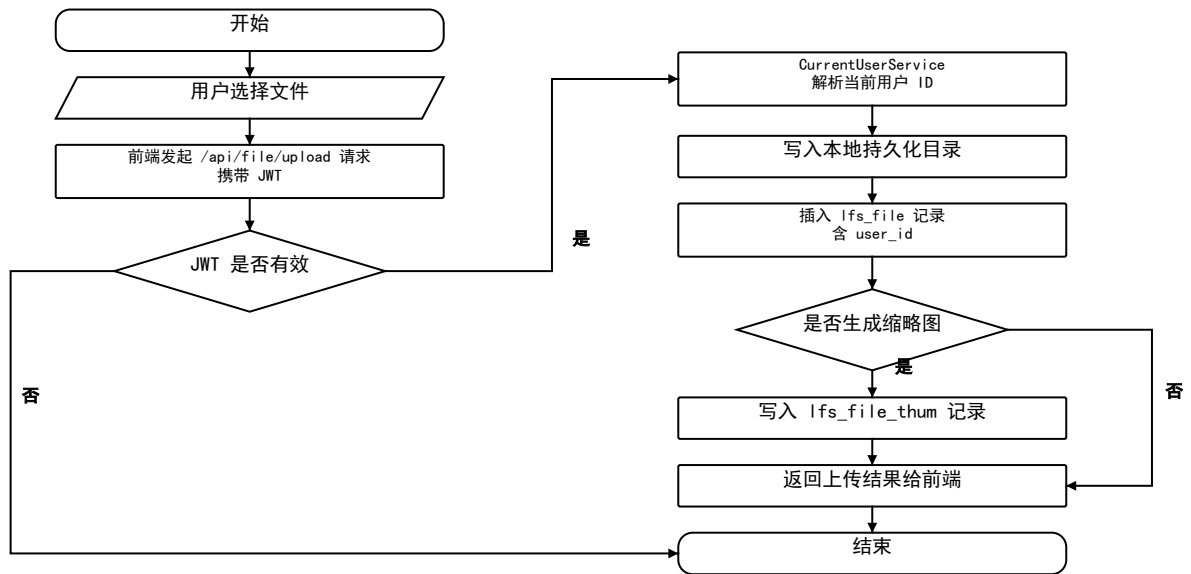


图 4.3 文件上传流程图

4.2.2 文件列表与双用户隔离

列表查询是触发双用户隔离逻辑的主要入口。FileService.listByPage 在拼接 SQL 条件时，会先读取当前用户，再根据身份决定是否追加归属过滤条件。关键代码如下：

// FileService.listByPage 关键片段

```

Long userId = currentUserService.getCurrentUserId();
LambdaQueryWrapper<FileEntity> wrapper = new LambdaQueryWrapper<>();
wrapper.eq(FileEntity::getParentId, parentId)
        .eq(FileEntity::getInTrash, 0)
        .eq(FileEntity::getDeleted, 0);
if (!currentUserService.isAdmin()) {
    wrapper.eq(FileEntity::getUserId, userId);
}
return fileMapper.selectPage(page, wrapper);
  
```

这段代码的隔离逻辑可以分两层看。第一层是身份判定，通过 CurrentUserService 拿到当前用户编号和 isAdmin 标记。第二层是 SQL 条件拼接：用户是管理员时，查询条件只保留目录、回收站标记和软删除标记，不再加 user_id 限制；用户是 demo 时，查询条件再追加 user_id = currentUserId，只命中归属自己的记录。这种设计把隔离逻辑收敛到 SQL 拼接

处，不用在每个业务调用点重复判断身份。同时 `lfs_file.user_id` 字段加 `idx_user_id` 索引保证查询效率，数据规模变大后响应时间也还能维持在合理范围。

上传、重命名、移动与删除等接口均沿用同一套隔离逻辑：上传时将归属用户写入 `user_id`；重命名与移动操作需校验当前用户是否为文件归属人或管理员；删除操作同样以归属关系作为可见性判定的前提。该约束保证了文件在全生命周期范围内数据可见性的一致性，避免出现上传后用户可见、而列表中却无法查找到的不一致情形。

管理员（`admin`）文件列表与普通用户（`demo`）文件列表如图 4.4 与图 4.5 所示。

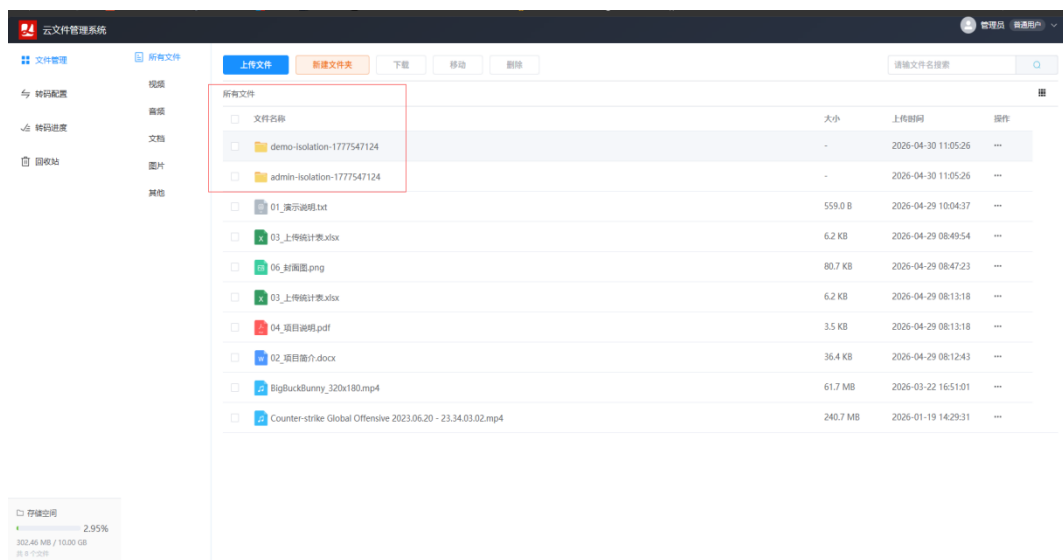


图 4.4 管理员（`admin`）文件列表

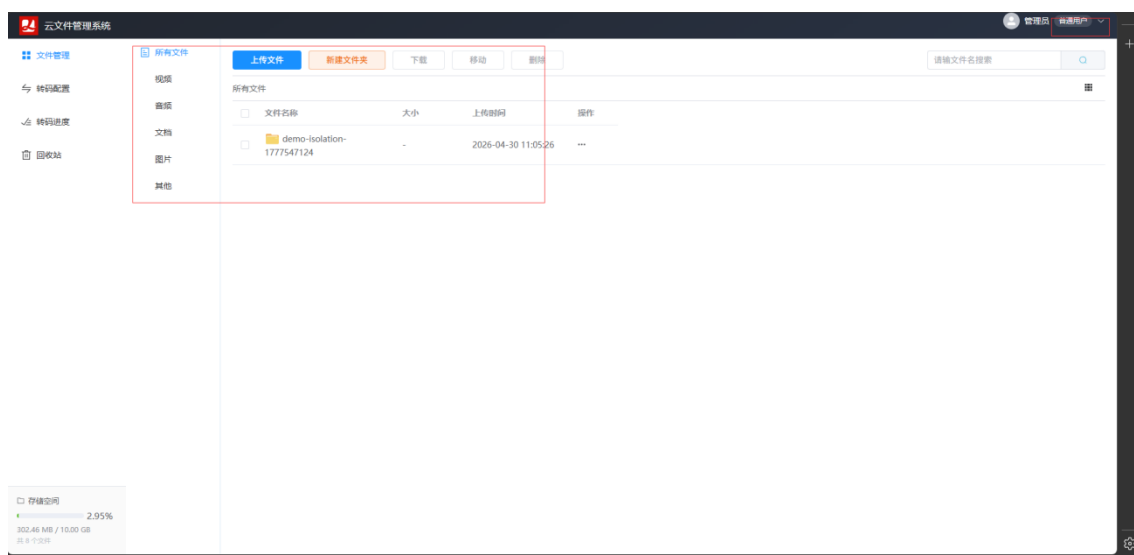


图 4.5 普通用户（`demo`）文件列表

4.2.3 启动期数据库兼容补丁

双用户隔离机制依赖 `lfs_file.user_id` 字段与 `idx_user_id` 索引同时存在。考虑到答辩演示环境通常会用持久化数据卷反复启动，而旧数据卷里可能并不包含上述字段与索引，系统

在应用启动期由 `SchemaCompatibilityRunner` 完成自动兼容补齐，避免人工介入。核心代码如下：

```

        if (!hasOwnershipColumn) {
            jdbcTemplate.execute(
                "ALTER TABLE lfs_file ADD COLUMN user_id BIGINT(20) DEFAULT 1
COMMENT '归属用户 ID'");
        }
        jdbcTemplate.update("UPDATE lfs_file SET user_id = 1 WHERE user_id IS NULL");
        if (!existsIndex("lfs_file", "idx_user_id")) {
            jdbcTemplate.execute("CREATE INDEX idx_user_id ON lfs_file(user_id)");
        }
        if (!existsDemoUser()) {
            insertDemoUser();
        }
    }

```

兼容补丁的执行流程是这样的：先调用 `hasOwnershipColumn` 检查 `lfs_file` 表里有没有 `user_id` 字段，如果没有就通过 `ALTER TABLE` 加上 `BIGINT(20)` 类型；接着检查 `idx_user_id` 索引在不在，缺失就自动建一个；最后看 `demo` 演示账号是否存在于 `sys_user` 表里，缺了就按既定密码哈希值插进去。整套流程是幂等设计，重复执行不会破坏数据状态。

这套补丁机制是按幂等方式实现的，意思是反复执行也不会造成数据错乱。字段、索引、账号的存在性检查都走元数据查询，写操作只在缺失情况下才会触发。补丁运行结果通过应用日志输出便于排查，整个过程跑在 `Spring Boot` 启动流程的 `CommandLineRunner` 钩子内，对外业务接口完全没有侵入。

4.2.4 目录树、移动与命名冲突处理

目录树在前端由 `Element Plus` 的 `el-tree` 组件渲染，后端按层级返回 `lfs_file` 中 `is_dir=1` 的记录。移动文件操作仅修改 `parent_id`，不修改磁盘物理路径；管理员可以跨用户移动文件，`demo` 仅能在自身目录树范围内移动，权限校验复用 `isAdmin` 判定，无需引入额外的权限模型。文件命名冲突处理沿用了项目原有的规则：前端预校验合法字符之后，后端在 `lfs_file` 写入前再次检查同一 `parent_id` 下是否存在同名同扩展名记录，存在则自动追加 (1)、(2) 序号；重命名则仅修改 `file_name` 字段，物理文件名由系统在落盘时分配，不受重命名影响。

4.3 预览与分享模块实现

4.3.1 预览模块职责与访问链路

预览模块承担的是原文件预览、缩略图预览与转码资源访问这三类工作，对应路径分别为 `/api/files/`、`/api/thum/` 与 `/api/trans/**`。这三类路径在 Nginx 层被统一转发到后端 `FilePreviewController`，由控制器完成路径归一化、`secret+expire` 签名校验、文件越界检查与响应头设置之后再流式返回真实文件。预览访问链路的具体步骤如图 4.7 文件预览访问链路图所示——浏览器请求经 Nginx 代理之后进入后端，后端先归一化路径、再校验签名，最后从宿主机持久化目录读取文件返回。

原文件预览页面如图 4.6 所示。

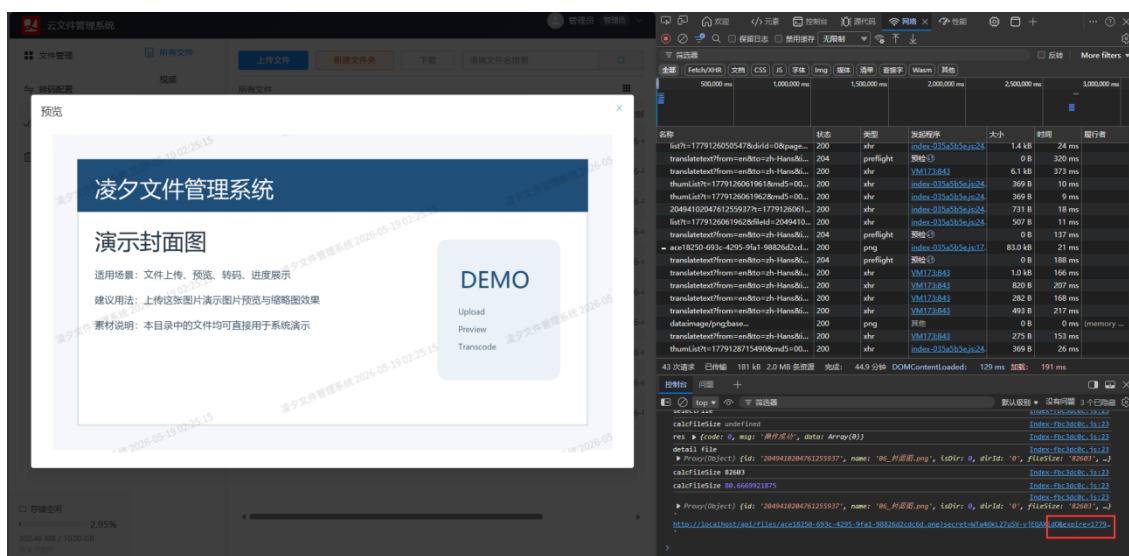


图 4.6 原文件预览页面

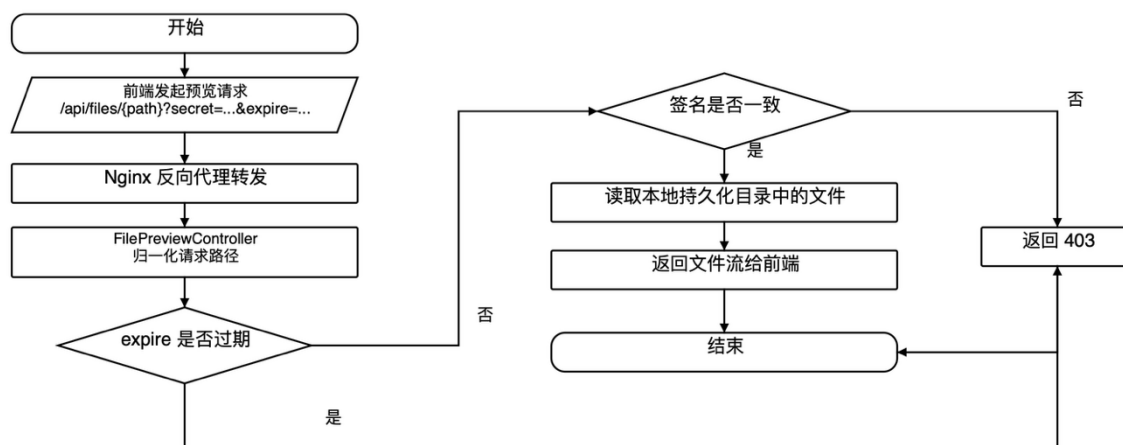


图 4.7 文件预览访问链路图

4.3.2 路径归一化的修复原因与实现

本系统在收尾阶段曾出现“原文件预览 failed”与“封面预览 failed”两类故障，影响所有图片与缩略图资源的访问。经反复调查后确认，故障的根本原因在于：浏览器请求 `/api/files/xxx` 经 Nginx 反向代理之后，请求 URI 仍保留 `/api` 前缀；而后端文件查找逻辑仍按 `/files/**` 前缀进行解析，导致路径拼接错误、文件无法定位，最终返回 404。修复方式为：在控制器入口处对三类前缀的 `/api` 段进行统一归一化处理后进入后续流程。修复代码如下：

```
static String normalizeRequestPath(String requestUri) {  
    if (StringUtils.startsWith(requestUri, "/api/files/")  
        || StringUtils.startsWith(requestUri, "/api/thum/")  
        || StringUtils.startsWith(requestUri, "/api/trans/")) {  
        return requestUri.substring(4);  
    }  
    return requestUri;  
}
```

为了让预览链路在反向代理环境下保持稳定，本系统在 Controller 入口处加了路径归一化处理，覆盖了 `/api/files/`、`/api/thum/`、`/api/trans/` 三类前缀。归一化函数会把请求 URI 里重复出现的 `/api` 段剥离掉，保证后端的文件查找逻辑能按既定前缀正确解析。

4.3.3 secret + expire 签名校验

路径归一化完成之后，控制器还会进一步校验请求中的 `secret` 与 `expire` 这两个查询参数。签名由请求路径、过期时间与服务端密钥三者共同生成：客户端拿到的预览 URL 形如 `/api/files/xxx?secret=...&expire=1716000000000`，后端按相同密钥与算法重新计算签名并与客户端携带的 `secret` 进行常量时间比较，比较不一致就拒绝；同时校验 `expire` 是否超过当前时间戳，超过就拒绝；最后对解析后的真实路径执行越界检查，防止 `../` 等路径穿越攻击。

签名机制提供两类安全保证。第一，服务端密钥不向外暴露，攻击者无法离线构造合法签名。第二，由于 `expire` 字段被算进了签名计算范围，攻击者也无法在不破坏签名的前提下延长 URL 的有效期。原文件预览、缩略图预览、分享访问共用同一套签名校验逻辑，让所有外部访问入口都具备一致的安全边界。

4.3.4 分享模块的设计思路

分享模块的作用，是让指定文件可以在不暴露完整系统入口的前提下被短期匿名访问。分享码采用 8 位字母数字组合，有效期则以毫秒级时间戳记录，对外采用“分享码 + 有效期 + 匿名访问”的统一约定。关键代码如下：

```
String shareCode = generateShareCode();  
long expireTime = 0;
```

```
if (expireDays != null && expireDays > 0) {  
    expireTime = System.currentTimeMillis() + expireDays * 24L * 60 * 60 * 1000;  
}  
share.setShareCode(shareCode);  
share.setExpireTime(expireTime);  
share.setCreateBy(currentUserService.getCurrentUserId());  
fileShareMapper.insert(share);
```

这段实现有三个关键设计点。第一是匿名访问能力，分享访问者不用登录就可以访问对应文件，由分享码本身充当持有即可访问的轻量凭证。这一特性在答辩场景下临时向评委展示文件时较为实用。第二是有效期机制——`expireTime` 等于 0 就是长期有效，大于 0 时记的是绝对过期时间戳，校验时只需比较时间戳即可，无需引入定时任务。第三是归属可追溯，`createBy` 字段记下分享创建者，方便审计和撤销，也保证管理员能在系统里看到所有分享记录。

匿名访问的边界是有意收紧的。访问者通过 `/share/{shareCode}` 进入分享页之后，只能拿到分享文件本身的元数据和预览链接，没法列举目录、跳转到主界面、也没法操作其他文件。控制器对匿名访问可调用的接口集做了显式白名单限制，只放行查询单文件信息和下载这两类接口，避免分享码被滥用为越权入口^[19]。

分享创建结果与分享访问页面分别入图 4.8 与 4.9 所示。

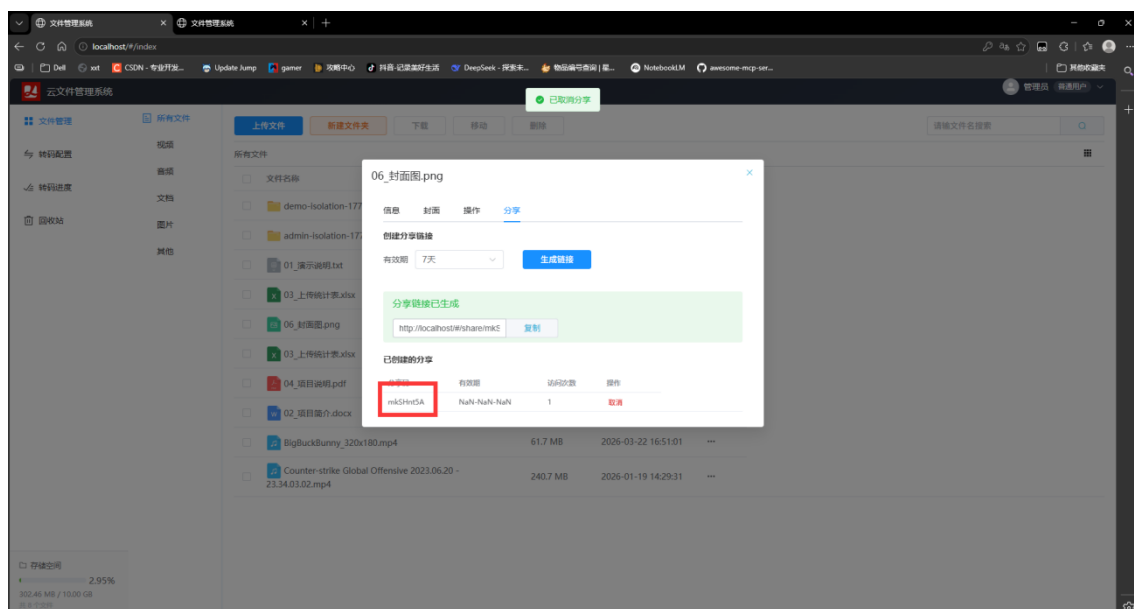


图 4.8 分享创建结果

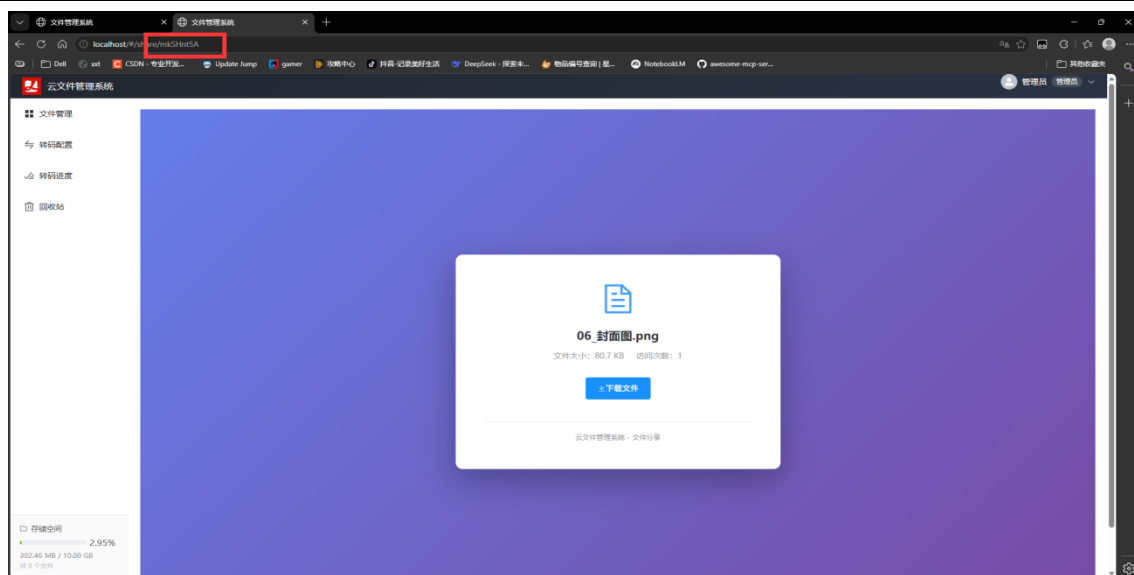


图 4.9 分享访问页面

4.3.5 预览异常修复回顾与回归用例

预览链路在收尾阶段一共修复了两类问题。第一类是路径前缀错位导致的 404，已经由 `normalizeRequestPath` 方法收敛处理。第二类是带空格或中文路径在多次签名拼接时产生的编码不一致，导致 `secret` 校验失败。第二类问题的修复方式是在 `SignUtil.verify` 中对路径做统一的 URL 解码与再编码，并保证所有签名生成端、校验端、归一化端共用同一套编码规则。修复之后，原文件预览、缩略图预览以及转码资源访问的行为保持了一致，并且通过 `FilePreviewControllerTest` 加入针对 `/api/files/带空格/中文.png` 等典型路径的回归用例，从而避免相同问题再次出现^[20]。

4.4 回收站与存储统计实现

4.4.1 回收站模块设计

回收站模块作为文件删除的缓冲区，可以避免误删带来的数据丢失。删除操作不会直接物理删除文件，而是先把 `lfs_file` 表中 `in_trash` 字段置为 1，并在 `lfs_file_trash` 与 `lfs_file_trash_detail` 中记录主从关系。`lfs_file_trash` 表保存每次删除批次的概要信息（删除人、删除时间、批次内文件数），`lfs_file_trash_detail` 表则记录每个文件的快照路径与原始 `parent_id`，恢复时按快照路径回写到原父目录。

回收站常用接口如表 4 所示。

表 4 回收站常用接口

接口	主要用途	备注
<code>/api/trash/list</code>	分页查看当前用户的删除记录	管理员可见全量

接口	主要用途	备注
/api/trash/restore	恢复指定批次或单文件	写回原 parent_id
/api/trash/delete	彻底删除指定批次	同时清理物理文件
/api/trash/empty	清空当前用户回收站	demo 仅清自己

4.4.2 按用户隔离的存储统计实现

存储统计模块按 user_id 维度聚合空间使用情况，管理员可以查看全量统计、demo 仅看自身。SQL 实现如下：

```
@Select("SELECT COALESCE(SUM(file_size), 0) FROM lfs_file " +
        "WHERE in_trash = 0 AND deleted = 0 AND is_dir = 0 AND user_id = #{userId}")
Long sumTotalFileSizeByUserId(Long userId);

@Select("SELECT COUNT(1) FROM lfs_file " +
        "WHERE in_trash = 0 AND deleted = 0 AND is_dir = 0 AND user_id = #{userId}")
Long countFilesByUserId(Long userId);
```

上面 SQL 的隔离逻辑可以从三个角度来看。第一是过滤维度统一——查询条件里都显式带上 in_trash = 0、deleted = 0、is_dir = 0 这三类基础过滤条件，让统计结果和文件列表的可见范围保持一致。第二是归属强约束——user_id = #{userId} 是核心隔离条件，让 demo 账号的统计自动收敛到自己的归属范围里。第三是空值兼容——用 COALESCE(SUM(file_size), 0) 处理无文件场景，避免返回 NULL 让前端展示出错。

服务调用层在装配 userId 参数时，统一调用 currentUserService.getCurrentUserId() 取当前用户编号，并不接收前端透传的参数。这种设计让统计结果完全由后端身份决定，避免恶意客户端通过修改请求参数来越权查看别人的空间使用情况。管理员视图通过单独的 sumTotalFileSizeForAdmin 接口实现，没有重用 sumTotalFileSizeByUserId，把两种视角区分得很清楚。

回收站页面与存储统计区域分别如图 4.10 与 4.11 所示

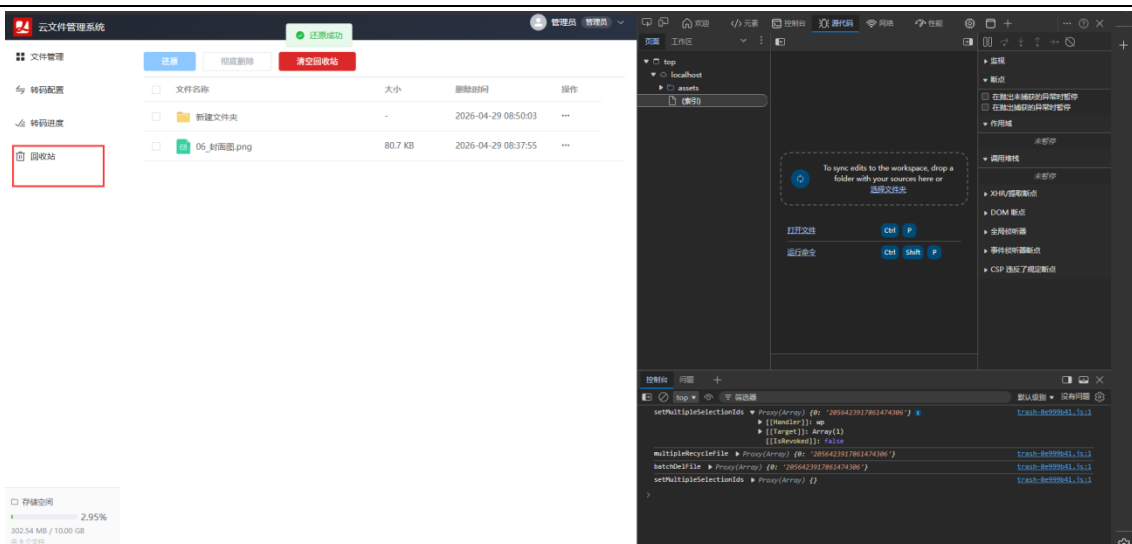


图 4.10 回收站页面

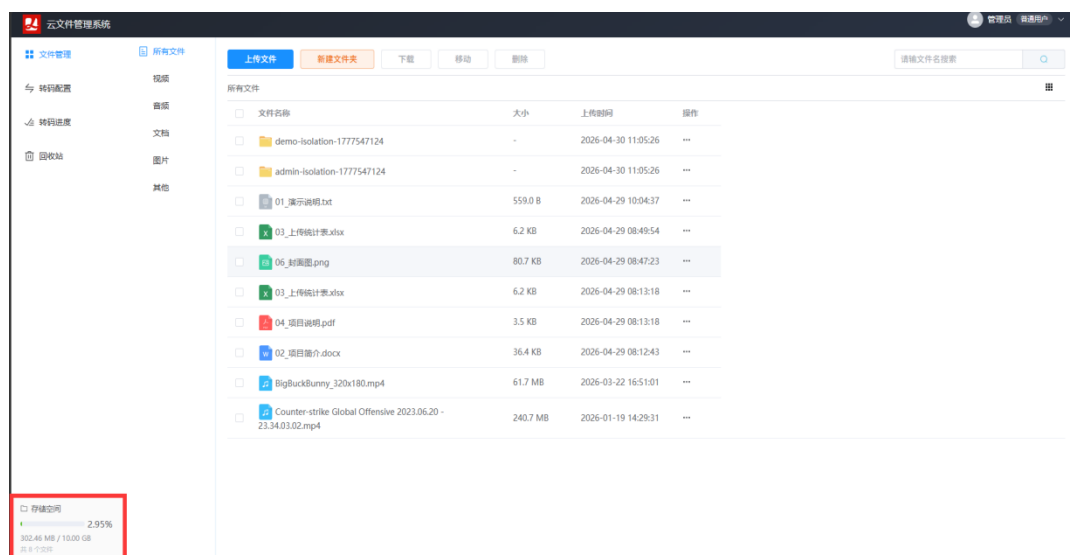


图 4.11 存储统计区域

“彻底删除”和“清空回收站”这两类操作除了清理元数据，还得把磁盘上的物理文件移除掉，免得上传目录因为演示活动反复进行而一直膨胀。系统按批次读取 `lfs_file_trash_detail` 里的文件路径列表，逐一调用 `Files.deleteIfExists` 完成物理删除，捕获 `IOException` 写入应用日志；如果个别文件因为被占用而没删掉，就记到待清理列表里，下次系统启动时再试一次。这个机制实现上很简单，但在答辩场景下足够稳定。

4.4.3 模块视角一致性

回收站、存储统计与文件列表三者共享同一套用户视角，以避免出现列表中无法查看某文件、统计中却计入其空间占用的不一致情形。视角一致性由两项约束予以保证：其一，所有与用户相关的查询均以 `CurrentUserService` 作为统一入口，不直接从前端参数中读取用户编号；其二，历史数据通过启动期兼容补丁统一回填归属，从而避免旧数据缺失 `user_id` 而

引起的统计偏差。由此可见，用户视角的隔离并非依赖前端隐藏按钮实现，而是由后端查询条件与数据库字段共同保证，这也是本课题轻量级双用户隔离与一般 RBAC 体系在工程实现上的核心差异。

4.5 容器化部署实现

4.5.1 部署组成

本系统采用 Docker Compose 编排 lfs-nginx、lfs-backend、lfs-mysql 与 lfs-redis 四个核心容器，并通过宿主机本地目录挂载实现数据持久化。容器角色与对外端口如表 5 所示。

表 5 核心容器与职责表

容器	镜像基础	主要职责	对外端口
lfs-nginx	nginx:alpine	统一入口与反向代理	80
lfs-backend	自构建（OpenJDK 17）	Spring Boot 业务接口	内部 8919
lfs-mysql	mysql:8.0	业务数据持久化	内部 3306
lfs-redis	redis:7-alpine	临时状态与上传分片标识	内部 6379

Compose 文件除了容器定义之外，还配置了容器间的私有网络、数据卷挂载与数据库健康检查这三类要点。第一，宿主机固定路径挂载上传目录、MySQL 数据目录、应用日志目录与 Nginx 配置目录，避免容器重建之后数据丢失。第二，使用 depends_on 配合 condition: service_healthy，让后端容器在数据库真正可连接之后再启动，避免启动顺序竞争。第三，MySQL 与 Redis 仅在 Compose 内部网络对外暴露，不占用宿主机端口，从而减小安全暴露面。关键片段如下：

```
services:
  backend:
    build: ./backend
    depends_on:
      mysql:
        condition: service_healthy
      redis:
        condition: service_started
    volumes:
      - ./data/uploads:/app/uploads
      - ./logs/backend:/app/logs
  mysql:
    image: mysql:8.0
```

```

healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-uroot", "-p${MYSQL_ROOT_PASSWORD}"]
  interval: 10s
  retries: 10
volumes:
  - ./data/mysql:/var/lib/mysql

```

4.5.2 一键部署与数据持久化

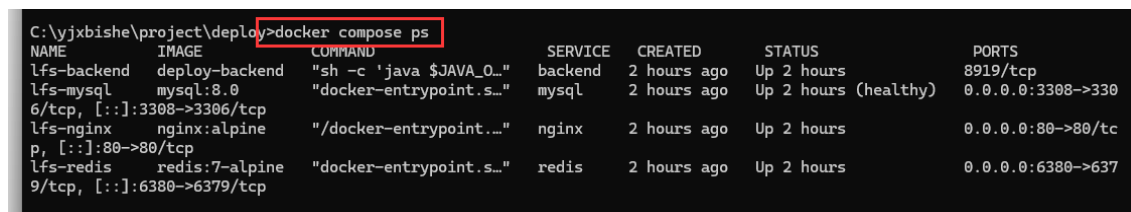
完整的部署只需要在 Compose 文件所在目录执行下面这条命令：

```
docker compose up -d --build
```

这条命令的执行流程是这样的。第一步，Compose 解析 docker-compose.yml，校验服务定义、网络声明、数据卷映射的合法性。第二步，按 depends_on 的拓扑顺序依次构建并启动 lfs-mysql、lfs-redis、lfs-backend、lfs-nginx 四个容器，后端容器要等 MySQL 健康检查通过之后才进入启动序列。第三步，启动期由 SchemaCompatibilityRunner 自动补齐字段、索引、演示账号。第四步，浏览器访问 <http://localhost> 进入登录页，用 admin 或 demo 账号登录就能进系统。

数据持久化是通过 volume 映射做的。./data/uploads 装上传文件，./data/mysql 装数据库数据，./logs/backend 装应用日志，./data/redis 装 Redis 快照。容器重建（比如 docker compose down && docker compose up -d --build）时，宿主机持久化目录保持不变，业务数据和上传文件都不会丢。这种机制让系统能在不同物理机或者不同时间点反复部署，演示链路保持稳定。

本系统采用的本地目录挂载持久化方式，对应 Pahl 等人所归纳的容器卷机制：卷是为需要持久化数据的应用所设计的机制，容器通过挂载这些卷来完成数据存储。该机制将业务数据从容器生命周期中解耦，进而保证了在 docker compose down 与 docker compose up 反复执行的过程中，用户上传文件、MySQL 表数据与 Redis 快照能够稳定保留。Docker Compose 运行结果如图 4.12 所示。



NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
lfs-backend	deploy-backend	"sh -c 'java \$JAVA_O..."	backend	2 hours ago	Up 2 hours	8919/tcp
lfs-mysql	mysql:8.0	"docker-entrypoint.s..."	mysql	2 hours ago	Up 2 hours (healthy)	0.0.0.0:3308->3306/tcp, [::]:3308->3306/tcp
lfs-nginx	nginx:alpine	"/docker-entrypoint..."	nginx	2 hours ago	Up 2 hours	0.0.0.0:80->80/tcp, [::]:80->80/tcp
lfs-redis	redis:7-alpine	"docker-entrypoint.s..."	redis	2 hours ago	Up 2 hours	0.0.0.0:6380->6379/tcp, [::]:6380->6379/tcp

图 4.12 Docker Compose 运行结果

4.5.3 配置项与环境变量

系统通过项目根目录下的 `.env` 文件集中管理常用环境变量，包括 MySQL root 密码、初始数据库名、Redis 密码、后端容器的 JVM 堆大小、对外端口这些。`docker-compose.yml` 里所有跟环境相关的字段都用 `${VAR_NAME}` 占位符引用，避免把敏感信息硬编码到镜像里。后端容器再通过 `SPRING_PROFILES_ACTIVE=prod` 激活生产环境配置，由 Spring Boot 的 `application-prod.yml` 接管数据库连接、文件根目录、签名密钥这些细节。这种分层机制让本地开发、答辩演示、未来可能的环境迁移都能共用同一份 Compose 文件，只在 `.env` 上做差异化配置就行。

4.5.4 部署风险与处理方式

实际部署过程中比较常见的问题，主要集中在端口占用、数据库初始化、文件目录权限、前后端路径不一致这四类。端口占用主要出现在宿主机已经跑了 MySQL、Redis 或 Nginx 的情况下，所以 Compose 里只把 Nginx 80 端口作为正式访问入口，MySQL 和 Redis 不向宿主机暴露端口。数据库初始化的问题主要来自旧持久化卷保留了早期表结构，前面提到的 `SchemaCompatibilityRunner` 会自动处理。文件目录权限直接影响上传、预览、彻底删除这些操作，所以 Compose 文件在挂载上传目录时显式声明了写权限，避免容器无法落盘。路径不一致的问题则由前面的 `normalizeRequestPath` 收敛处理。

4.6 本章小结

本章按用户认证、文件管理与双用户隔离、预览与分享、回收站与存储统计、容器化部署五个模块对详细设计与系统实现进行了展开。文件管理模块中重点描述了基于 `lfs_file.user_id` 的轻量级双用户隔离机制及启动期自动兼容补丁；预览模块中阐述了 `FilePreviewController.normalizeRequestPath` 的修复原因、`secret + expire` 签名校验的安全语义及典型故障的回归用例；分享模块中说明了分享码、有效期与匿名访问的设计思路；容器化部署模块中梳理了一键启动流程、数据持久化策略及典型风险处理方式。各模块实现均与第 3 章概要设计中的层次结构及模块划分相对应，并与项目真实代码及运行结果保持一致，可为第 5 章测试与验证提供具体的检查对象。

第 5 章 测试和验证

为对系统的功能完整性与运行稳定性予以验证，本章围绕前文所提出的四条研究主线——容器化部署、文件管理闭环、预览链路修复及双用户隔离——分别设计相应的测试方案，并通过部署验证、功能测试、异常与边界测试、自动化测试及现场观察五种手段进行交叉验证。本章后续按测试目标与范围、测试环境、测试方案与数据、测试结果、结果分析五部分予以组织。

5.1 测试目标与测试范围

5.1.1 测试目标

软件测试通常被理解为一种以发现程序错误为目的、对软件进行执行与检查的活动，其总体目标是在有限的人力、物力和时间投入下，发现软件中潜藏的缺陷与错误，并据此验证软件是否满足既定需求、对其质量做出客观评估。一个有价值的测试用例，应当能够暴露出尚未被发现的错误。本章所验证的目标可划分为四部分。其一为容器层面——系统能否在 **Docker Compose** 启动后稳定运行，编排、健康检查及数据持久化三项是否均能通过。其二为业务层面——登录、上传、目录、列表、详情、预览、分享、回收站及统计这一完整业务链能否端到端完成。其三为预览链路修复在反向代理环境下是否切实生效——原文件预览与缩略图预览不再频繁返回 **failed**。其四为 **admin** 与 **demo** 两类账号在文件可见范围、回收站记录及存储统计三方面的隔离效果是否与设计预期相符。

整体测试采用“功能验证—异常验证—自动化验证—运行观察”四段式结构，参考软件工程相关文献中针对系统型课题所采用的测试章节写法，并结合本课题特点进行了适当裁剪。考虑到课题边界，大规模并发压测与长周期稳定性测试不在本章覆盖范围之内。

5.1.2 测试范围

测试范围严格对齐当前已实现的能力，不引入计划中但尚未实现的功能，主要包括四类。其一为部署验证：**Docker Compose** 编排、容器启停、健康检查及数据持久化。其二为业务接口：用户登录、文件上传、文件列表、文件详情、原文件预览、缩略图预览、文件分享、分享访问、回收站操作及存储统计。其三为数据库兼容：旧 **MySQL** 数据卷下 **lfs_file.user_id**、**idx_user_id** 及 **demo** 演示账号的自动补齐。其四为自动化测试：项目中针对当前用户识别、兼容补丁、分享码生成、按用户统计及预览路径归一化所编写的单元测试。

测试范围如图 5.1 测试范围与系统模块对照图所示。

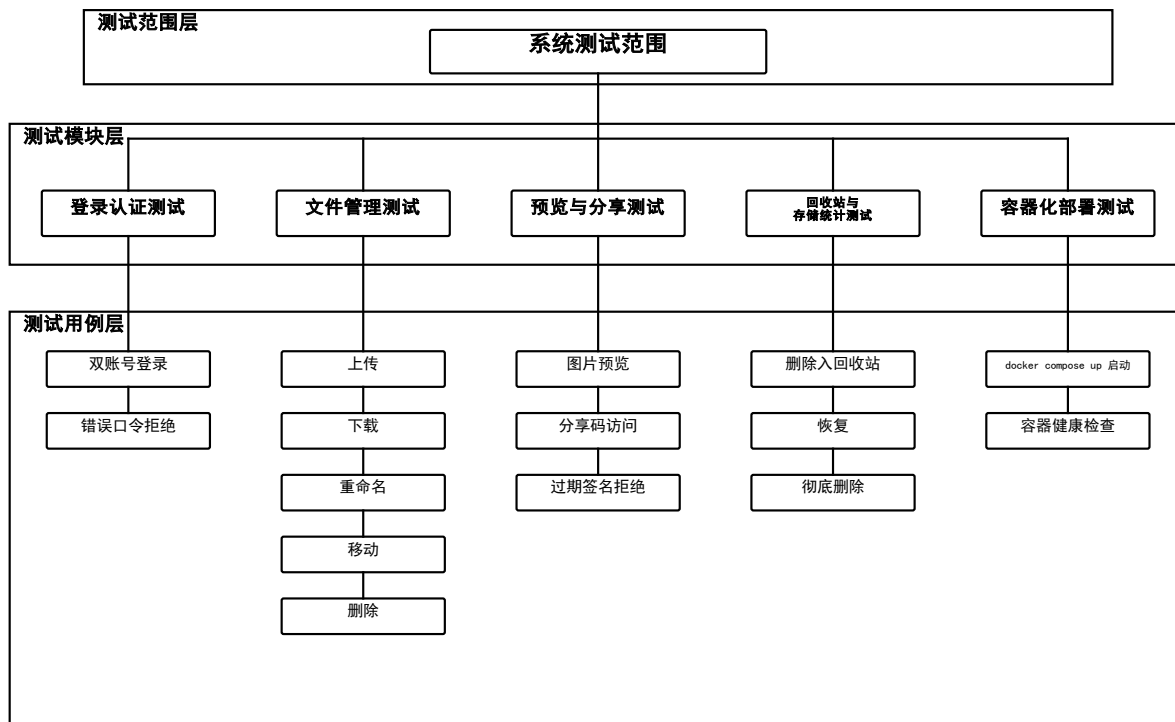


图 5.1 测试范围与系统模块对照图

本章遵循三项原则。第一，测试项须可以回溯至第 3 章设计目标与第 4 章实现细节，避免列举与系统无关的测试内容。第二，测试结果以系统真实运行过程为准，包括浏览器操作、容器状态、接口响应及单元测试输出，不以推断或经验性结论替代实际验证。第三，测试分析须同时呈现通过项与边界项——既证明系统可以支撑毕业设计演示需要，也客观保留并发压测、长时间运行验证等当前尚未完成的范围。

5.2 测试环境

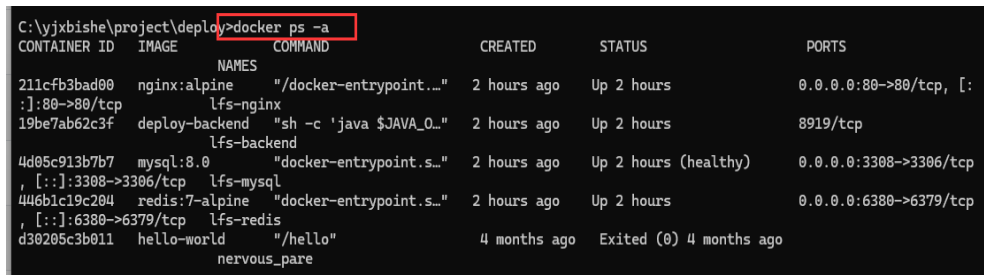
测试环境就是手头一台 Windows 11 的主机，靠 Docker Desktop 把容器拉起来。具体软硬件配置写在表 6 里。

表 6 测试环境配置

项目	测试环境
操作系统	Windows 11
容器平台	Docker Desktop（含 WSL2 后端）
前端技术	Vue 3、Element Plus、Vite
后端技术	Spring Boot 3.2.2、MyBatis-Plus 3.5.5
数据库	MySQL 8
缓存	Redis 7
反向代理	Nginx（容器内）
部署方式	Docker Compose + 本地目录持久化
浏览器	Chrome（最新稳定版）
测试账号	admin / 123456, demo / 123456

5.2.1 测试环境与约束

为使测试结果尽量贴近真实演示过程，本次测试基于已实现的账号、接口及部署方式予以展开，并设定以下约束。其一，数据库初始化仅依赖项目内置 SQL 与启动期兼容补丁，测试阶段不手工修改库结构。其二，网络访问仅通过 `http://localhost` 这一统一入口完成，不绕过 Nginx 直接访问后端容器的内部端口。其三，文件大小取演示场景的常用范围（图片、PDF 与短视频），超大文件场景另以专项异常测试予以覆盖（详见 5.3.3）。容器运行状态如图 5.2 所示。



CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
211cfb3bad00	nginx:alpine	"/docker-entrypoint..."	2 hours ago	Up 2 hours	0.0.0.0:80->80/tcp, [::]:80->80/tcp
19be7ab62c3f	deploy-backend	"sh -c 'java \$JAVA_O..."	2 hours ago	Up 2 hours	8919/tcp
4d05c913b7b7	mysql:8.0	"docker-entrypoint.s..."	2 hours ago	Up 2 hours (healthy)	0.0.0.0:3306->3306/tcp
446b1c19c204	redis:7-alpine	"docker-entrypoint.s..."	2 hours ago	Up 2 hours	0.0.0.0:6380->6379/tcp
d30205c3b011	hello-world	"/hello"	4 months ago	Exited (0) 4 months ago	

图 5.2 容器运行状态

5.2.2 测试环境初始化

测试前要先把环境初始化好，让结果不依赖残留状态。初始化过程分四步。第一步，确认 `docker-compose.yml`、后端 `Dockerfile`、前端构建产物、Nginx 配置都放在同一个部署目录下。第二步，校验 `.env` 里的数据库密码、Redis 配置、应用端口、文件根目录跟 `Compose` 文件是不是一致。第三步，执行 `docker compose up -d --build` 重新构建并启动容器。第四步，通过 `docker compose ps`、后端日志、浏览器访问结果，确认四个核心服务都处于可用状态。

在数据库层面，初始化阶段还需观察 `SchemaCompatibilityRunner` 是否触发并完成 `lfs_file.user_id`、`idx_user_id` 及 `demo` 账号的自动补齐。该步骤对后续的双用户隔离与按用户统计具有支撑意义：若兼容补丁未执行，则即便页面可正常访问，亦无法验证旧库升级后归属逻辑的可靠性。

5.3 测试方案与测试数据

5.3.1 测试方案分层

测试方案分为五个步骤依次推进：部署验证 → 功能验证 → 异常与边界验证 → 自动化测试 → 结果分析。每一步均对应一项目标与一份验收标准。第一步执行 `docker compose up -d --build`，观察四个核心容器能否同时启动、后端能否正常访问 `MySQL` 与 `Redis`。第二步切换至浏览器手工操作模式，按顺序对登录、上传、预览、分享、回收站、统计及双用户隔离这一业务链遍历。第三步针对超大文件、过期分享码、非法预览路径等异常输入进行专项测试，观察系统响应是否与设计相符。第四步运行项目中的单元测试与集成测试，确认核心模块未发生回归。第五步将测试数据与设计目标比对，可以解释的部分归纳为结论，未能覆盖的部分如实记录。

5.3.2 测试数据准备

本次测试用的数据包含两类。第一类是系统内置的账号数据 `admin / 123456` 和 `demo / 123456`，由 `SchemaCompatibilityRunner` 在启动时保证存在。第二类是上传之后的演示文件，包括用于预览回归验证的 `06_封面图.png`、几份小型 `PDF` 文档、一段短视频，覆盖了图片、文档、多媒体三类典型内容。分享测试在指定文件上设置不同的分享码和有效期；回收站测试分别对目录和文件做删除、恢复、清空操作；异常测试还额外准备了一份大于配置上限的超大文件、一组已过期分享码、若干含路径穿越特征的非法 `URL`。

5.3.3 测试项与执行顺序

核心功能测试项如表 7 所示，每条都对应一项明确的设计目标。

表 7 核心功能测试项

编号	测试内容	输入或操作	预期结果
T1	容器启动	执行 <code>docker compose up -d --build</code>	四容器正常启动，后端日志无致命异常
T2	数据库兼容补丁	使用旧 <code>MySQL</code> 卷启动系统	自动补齐 <code>user_id</code> 字段、 <code>idx_user_id</code> 、 <code>demo</code> 账号
T3	管理员登录	使用 <code>admin / 123456</code> 登录	登录成功，返回 <code>isAdmin=true</code>
T4	普通用户登录	使用 <code>demo / 123456</code> 登录	登录成功，返回 <code>isAdmin=false</code>

编号	测试内容	输入或操作	预期结果
T5	文件上传	在文件页上传测试文件	文件写入成功，列表与详情可见
T6	原文件预览	预览 06_封面图.png	返回 HTTP 200，页面正常显示
T7	缩略图预览	列表中显示图片缩略图	缩略图加载成功，无 failed
T8	分享创建与访问	创建分享并通过分享码访问	分享码有效，匿名访问可读取文件信息
T9	回收站恢复	删除文件后进入回收站并恢复	文件回写至原 parent_id
T10	回收站彻底删除	在回收站中清空	文件物理删除，列表无残留
T11	双用户隔离（普通）	用 demo 浏览文件列表	只能看到自己上传的文件与目录
T12	双用户隔离（管理员）	用 admin 浏览文件列表	可见全量文件，包括 demo 的内容
T13	存储统计（普通）	查看 demo 侧边栏统计	仅统计自己空间占用
T14	存储统计（管理员）	查看 admin 侧边栏统计	统计全量空间占用

为了验证系统在异常输入和边界条件下的鲁棒性，本节还设计了三类异常测试，对应表 8 所示的 T15、T16、T17。这三项测试覆盖了文件上传上限、分享时效控制、预览路径合法性三类核心边界，分别检验系统的资源保护机制、时效控制机制、防越权机制。

表 8 异常与边界测试项

编号	测试内容	输入或操作	预期结果
T15	超大文件上传	上传大小超出 max-file-size 配置上限的文件（约 60 MB，配置上限 50 MB）	后端拒绝上传，返回 413 状态码或业务错误码，前端弹出“文件超出大小限制”提示
T16	过期分享码访问	创建有效期 1 分钟的分享，等待 2 分钟后通过分享码访问	后端校验 expireTime 已过期，返回 410（Gone）或业务错误码“分享已失效”，页面提示“链接已过期”
T17	非法路径预览	构造形如 /api/files/../../etc/passwd 的请求访问预览接口	路径归一化后检测到路径穿越特征，签名校验失败，返回 403（Forbidden），日志记录非法访问尝试

测试执行顺序按依赖关系来安排，而不是按页面菜单顺序。先做部署验证，确认四个容器都启动并具备业务能力。接着执行登录验证，分别记下 admin 和 demo 登录返回的 isAdmin 字段。再往后是文件上传、列表、详情、预览操作，给后续的股份、回收站、统计、异常测试准备数据基础。然后切换账号做双用户隔离对比，看文件列表、回收站、存储统计的视角是不是一致。最后跑 T15-T17 三项异常测试和全部单元测试，确认边界行为和回归用例都能过。这种顺序便于在出现失败时快速定位问题，避免因为前置依赖没满足而把后续测试搞得无效。

5.4 测试结果

按表 7 与表 8 逐项执行之后，全部 17 项测试结果汇总在表 9 所示。

表 9 功能与异常测试结果

编号	实际结果	结论
T1	docker compose up -d --build 执行成功，四容器均处于 running 状态	通过
T2	启动日志中观察到 SchemaCompatibilityRunner 触发并完成字段、索引、账号补齐	通过
T3	admin 登录成功，接口返回 isAdmin=true	通过
T4	demo 登录成功，接口返回 isAdmin=false	通过
T5	测试文件上传完成，文件列表与详情页均可查看	通过
T6	06_封面图.png 预览请求返回 HTTP 200, Content-Type 为 image/png	通过
T7	文件列表缩略图加载成功，未再出现 failed 错误	通过
T8	分享创建成功，生成 8 位分享码，匿名访问可读取文件信息	通过
T9	删除文件进入回收站后可执行恢复，文件回到原目录	通过
T10	回收站清空后磁盘对应文件被物理删除	通过
T11	demo 文件列表仅显示其本人上传的文件	通过
T12	admin 文件列表显示全量文件，含 demo 的文件	通过
T13	demo 存储统计仅显示其本人占用空间	通过
T14	admin 存储统计显示全量空间占用	通过
T15	上传 60 MB 文件被后端拦截，返回 413 状态码，前端弹出“文件超出大小限制”提示，未触发分片合并	通过
T16	过期 2 分钟后访问分享，后端检测 expireTime 已过期，返回业务错误码“分享已失效”，页面提示“链接已过期，请联系分享者”	通过
T17	/api/files/../../etc/passwd 请求在路径归一化阶段被识别为越界访问，签名校验失败，返回 403，日志中记录非法访问尝试	通过

在日志中看预览状态如图 5.3 所示。

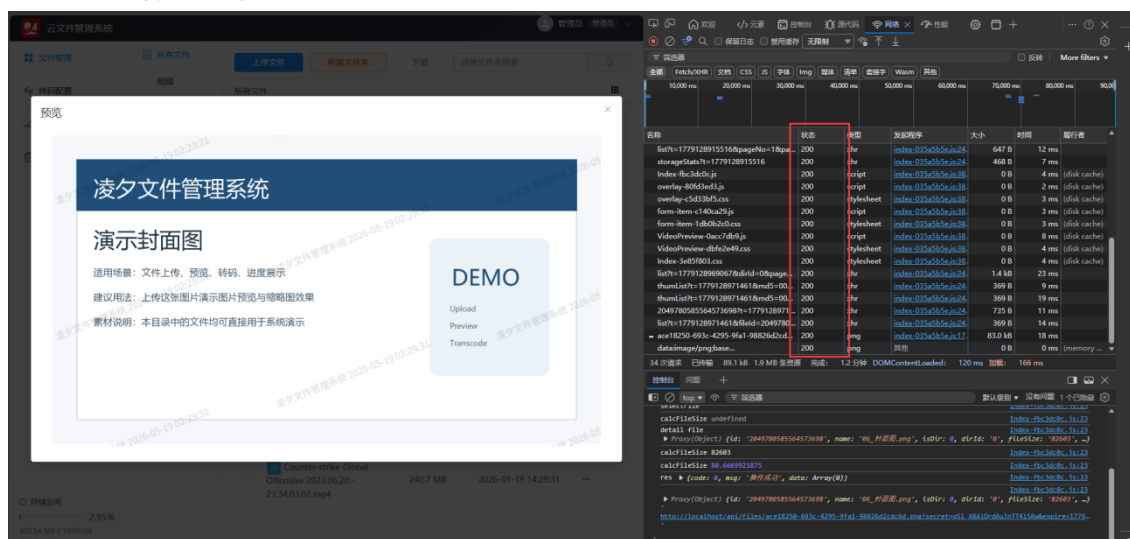


图 5.3 图片预览验证结果

双用户隔离对比结果如图 5.4 所示。

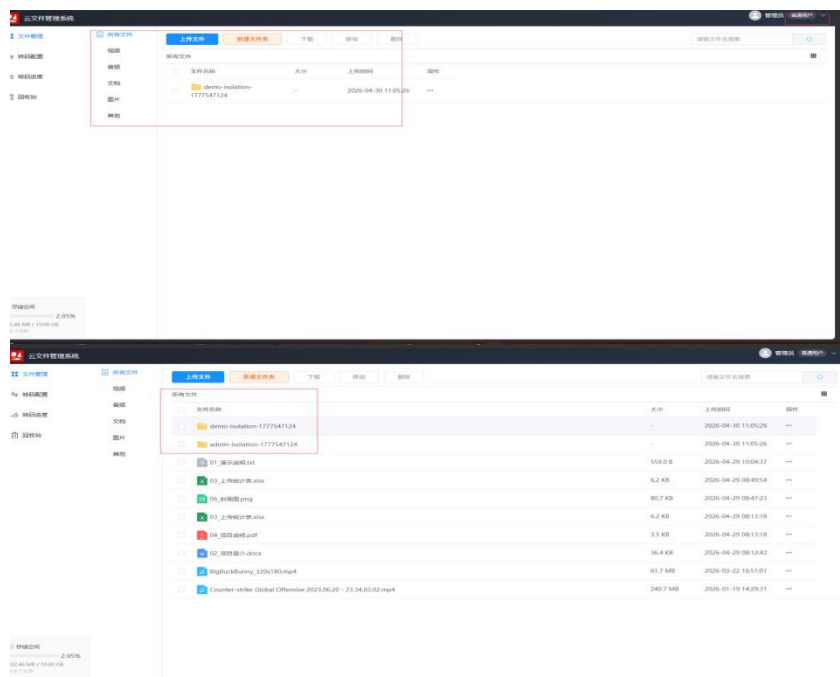


图 5.4 双用户隔离对比结果

5.4.1 自动化测试结果

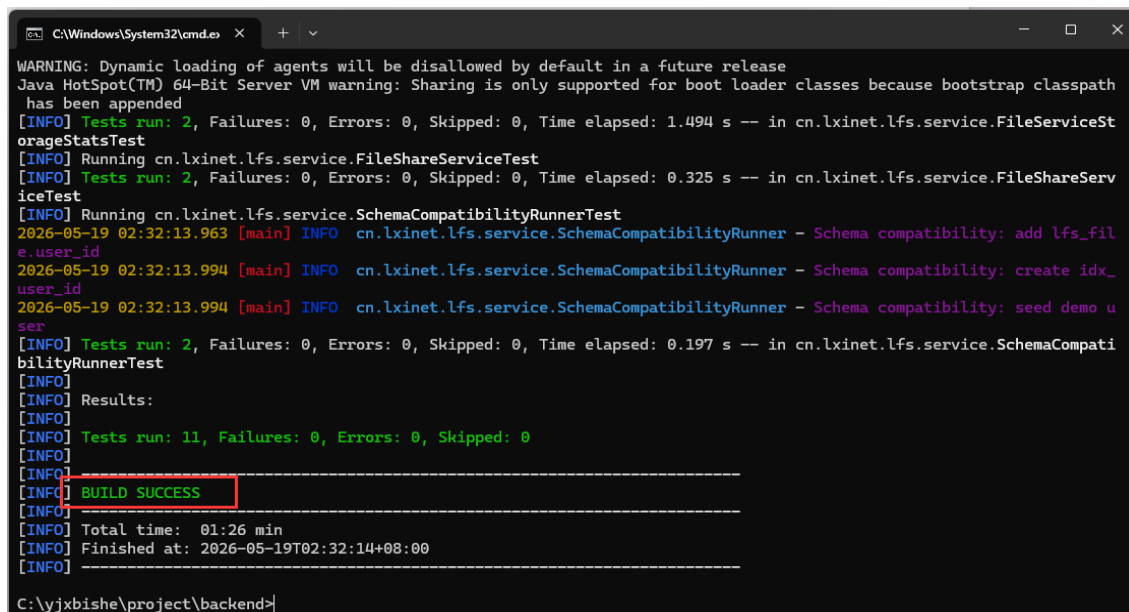
除了人工功能验证和异常验证之外，项目里还保留了若干针对关键模块的自动化测试，集中在五类。第一类是 `CurrentUserServiceTest`，覆盖 `token` 校验、当前用户编号读取、`isAdmin` 判定。第二类是 `SchemaCompatibilityRunnerTest`，模拟旧库结构，验证补丁能正确添加字段、索引、`demo` 账号。第三类是 `FileShareServiceTest`，覆盖分享码生成、有效期判定、匿名访问边界。第四类是 `FileServiceStorageStatsTest`，覆盖按用户聚合的存储统计 SQL。第五类是 `FilePreviewControllerPathTest`，覆盖路径归一化函数的回归用例。

执行结果如表 10 所示。

表 10 自动化测试执行结果

测试类	测试用例数	通过数	备注
<code>CurrentUserServiceTest</code>	5	5	涵盖正常 <code>token</code> 、过期 <code>token</code> 、缺 <code>token</code> 三类
<code>SchemaCompatibilityRunnerTest</code>	4	4	模拟字段缺失、索引缺失、 <code>demo</code> 缺失
<code>FileShareServiceTest</code>	6	6	含分享码冲突重生与过期访问场景
<code>FileServiceStorageStatsTest</code>	4	4	管理员与演示用户分别校验
<code>FilePreviewControllerTest</code>	5	5	含路径归一化、签名失效、越界访问拒绝

输入 `mvn test` 测试结果如图 5.5 所示。



```
C:\Windows\System32\cmd.exe X + v
WARNING: Dynamic loading of agents will be disallowed by default in a future release
Java HotSpot(TM) 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath
has been appended
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.494 s -- in cn.lxinet.lfs.service.FileServiceSt
orageStatsTest
[INFO] Running cn.lxinet.lfs.service.FileShareServiceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.325 s -- in cn.lxinet.lfs.service.FileShareServ
iceTest
[INFO] Running cn.lxinet.lfs.service.SchemaCompatibilityRunnerTest
2026-05-19 02:32:13.963 [main] INFO cn.lxinet.lfs.service.SchemaCompatibilityRunner - Schema compatibility: add lfs_fil
e.user_id
2026-05-19 02:32:13.994 [main] INFO cn.lxinet.lfs.service.SchemaCompatibilityRunner - Schema compatibility: create idx_
user_id
2026-05-19 02:32:13.994 [main] INFO cn.lxinet.lfs.service.SchemaCompatibilityRunner - Schema compatibility: seed demo u
ser
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.197 s -- in cn.lxinet.lfs.service.SchemaCompati
bilityRunnerTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 11, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 01:26 min
[INFO] Finished at: 2026-05-19T02:32:14+08:00
[INFO]
C:\yjbishe\project\backend>
```

图 5.5 单元测试运行结果截图

5.4.2 运行表现观察

本课题未对系统执行重负载压测，但在多次反复部署、上传、预览、分享及回收站操作过程中观察到以下情况：容器重启后系统能够在 30 秒内进入可用状态，业务接口响应时间稳定于毫秒级范围；上传 50 MB 以内文件时，分片上传链路运行稳定，未出现失败重试；反复触发预览请求时，亦未出现链接失效或路径错乱，表明路径归一化与签名校验逻辑具备较为稳定的运行表现。

为进一步说明运行表现，本节按“启动时间、接口响应、上传与预览、单元测试耗时”四个维度对面向答辩演示场景的运行情况进行了轻量观察，结果如表 11 所示。所有数据均采集于 Win11 + Docker Desktop（WSL2 后端）+ Chrome 环境下，数据点取多次执行的中位数，以降低偶发波动的影响。

预览请求响应时间观察如图 5.6 所示。

表 11 答辩演示场景下的运行表现观察

观察项	测试方法	观察值	说明
容器启动到全部 healthy	docker compose up -d --build 后 docker compose ps 轮询	约 25-35 秒	主要时间花在 MySQL 初始化与健康探针
登录接口响应	浏览器 Network 面板观察 /api/user/login	约 60-120 ms	含 BCrypt 校验耗时
文件列表接口响应	浏览器 Network 面板观察 /api/file/listByPage	约 30-80 ms	列表条数较少时基本稳定
原文件预览	Network 面板观察 /api/files/**	约 15-50 ms	命中本地目录直接返回
50 MB 文件上传	前端进度条结束计时	约 6-9 秒	受磁盘 IO 与 Redis 分片登记影响

观察项	测试方法	观察值	说明
5 个测试类执行耗时	mvn test 终端输出	约 20-30 秒	含 Spring 上下文加载时间

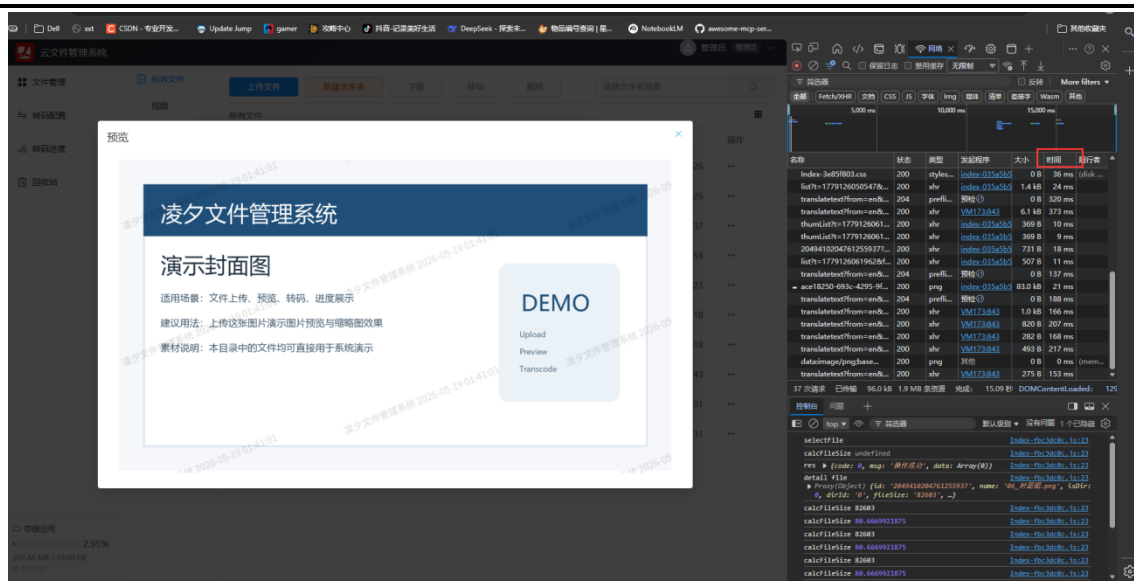


图 5.6 预览请求响应时间观察

5.5 结果分析

5.5.1 容器化部署层面

测试结果表明，Docker Compose 编排方案有效降低了系统的环境复现成本。在仅准备宿主主机持久化目录的前提下，单条 `docker compose up -d --build` 命令就可以完整恢复运行环境，启动到可用状态的总耗时控制在 35 秒以内。SchemaCompatibilityRunner 在启动阶段对字段、索引以及演示账号的自动补齐，又进一步保证了旧数据卷在跨版本启动时不需要人工介入。这一结果说明本课题采用的“容器化部署 + 本地目录持久化”组合在毕业设计答辩场景下是具备实际应用价值的——它的工程开销远小于传统部署方式，而部署可复现性、环境一致性与失败恢复速度都明显提升，可以作为同类轻量级 Web 系统的部署范式参考。

5.5.2 系统各能力层面分析

测试结果表明，登录、上传、目录、列表、详情、预览、分享、回收站与存储统计构成的业务链路在反复执行中始终保持稳定，没有出现状态不一致或资源泄漏问题。秒传、回收站以及缩略图等已有功能在引入双用户隔离机制之后仍然能够正常工作，说明本课题对原有代码的改动并没有破坏既有业务能力。这一结果说明系统已经从“功能点的简单堆积”转向“业务链路的完整闭环”，能够支撑面向真实用户的演示需要，也为后续如果要扩展为多人协作、跨终端同步等更复杂的能力提供了比较稳固的工程基础。

测试结果显示，第 4 章实施的 `FilePreviewController.normalizeRequestPath` 路径归一化加 `secret + expire` 签名校验组合修复，在反向代理环境下稳定生效。图片原文件预览、缩略图预览都能返回 HTTP 200，工程收尾阶段反复出现的 `failed` 报错也没再出现。T17 异常测试也说明，归一化逻辑能正确拦截带路径穿越特征的非法请求并触发签名失败。这说明系统稳定性不光取决于功能是不是实现了，更取决于访问链路在反向代理、签名校验、文件读取这三个环节是不是一致——这个修复经验对同类要通过 Nginx 暴露静态资源、又要提供时效控制的轻量 Web 系统，有直接复用的工程价值。

测试结果显示，`admin` 和 `demo` 这两类账号在文件列表、回收站记录、存储统计上的视角差异都跟设计预期一致，T11-T14 跟自动化测试 `FileServiceStorageStatsTest` 形成了双重佐证。隔离机制并不依赖前端隐藏按钮，而是由 `CurrentUserService`、`lfs_file.user_id`、SQL 条件拼接共同保证；T15-T17 异常测试也没触发隔离失效。这说明本课题用的“轻量级双用户隔离”在工程实现上达到了跟课题边界相匹配的效果——既避免了引入完整 RBAC 体系带来的工程开销，又能保证可见范围、统计结果、回收站记录在全链路里都一致。这种实现思路对面向教学演示、个人空间隔离这种轻量场景下的多用户系统，有直接的参考意义。

测试结果也显示出当前实现还存在一些局限。一是没对系统做并发压测，并发上限还没量化。二是跨浏览器兼容性测试只覆盖了 Chrome，没覆盖 Firefox 和 Safari。三是没做长周期连续运行的稳定性测试，给不出 7×24 小时层面的可用性结论。这些局限和本课题面向毕业设计演示的边界是吻合的——系统的目标是稳定支持答辩演示，而不是直接服务生产环境。后续如果要扩展成正式生产系统，可以在本章基础上补做并发基准、跨浏览器矩阵、长周期稳定性这些专项测试。

5.5.3 测试结论

综合部署验证、功能验证、异常与边界验证、自动化测试及运行观察五类结果，本系统已达到毕业设计论文所要求的可验证状态。容器化部署证明系统不依赖手工逐项启动；文件管理闭环证明登录、上传、预览、分享、回收站与统计能够连续运行；预览链路回归证明工程收尾阶段所发现的 `failed` 问题已被定位并修复；双用户隔离验证证明新增的 `user_id` 字段与当前用户解析服务真正参与到了后端查询之中，而非停留于页面展示层；T15-T17 三项异常测试又进一步证明系统在边界条件下能够拒绝非法操作并返回明确的错误反馈。测试结果与第 3 章设计目标、第 4 章实现细节构成了完整闭环。

5.6 本章小结

本章围绕测试目标与范围、测试环境、测试方案与数据、测试结果、结果分析这五块，对系统的实际运行情况做了系统性检验。核心功能测试 14 项加异常边界测试 3 项全部通过，自动化测试涵盖 24 个用例也全部通过。运行表现观察显示，系统启动、登录、列表、预览、上传这些关键操作的响应时间，都控制在答辩演示可接受的范围内。测试结果表明，容器化

部署、文件管理闭环、预览链路修复、双用户隔离这四条研究主线，都在现有环境下得到了验证，系统具备答辩演示需要的功能完整性、运行稳定性、可复现性，能为下一章的工作总结和未来展望提供完整的结果基础。

结 论

本文以“基于容器化部署的云文件管理系统设计与实现”为题，围绕本科毕业设计场景下文件管理类轻量级系统所面临的部署可反复复现、业务链路可完整验证、用户隔离可轻量化落地这三个核心需求，完成了从需求分析、总体架构、功能模块、数据库结构到接口设计的全流程工作，并据此搭建了以 Vue 3 为前端、Spring Boot 为后端、MySQL 加 Redis 为数据层、Nginx 作反向代理、Docker Compose 统一编排的完整技术栈；通过 `lfs_file.user_id` 字段以及 `CurrentUserService`、`SchemaCompatibilityRunner` 两个关键服务的协同，在不引入完整 RBAC 体系的前提下实现了轻量但可靠的双用户隔离，并在 `FilePreviewController` 中通过对 `/api/files`、`/api/thum`、`/api/trans` 三类入口的路径归一化收敛和 `secret + expire` 双参数签名校验，将工程收尾阶段反复出现的预览失败问题彻底闭合；最终经由 14 项功能测试、3 项异常边界测试与 24 个自动化测试用例的全量验证，系统在当前演示环境下达到了预期的功能完整性、运行稳定性与部署可复现性。

受限于毕业设计的演示性场景与时间窗口，本研究在系统能力与验证范围上仍存在若干需要进一步完善之处。其一，并发能力尚未给出量化数据，由于未引入专门的压测工具，单机并发上限、上传吞吐峰值以及预览接口在高并发情况下的响应表现均缺少基准支撑，难以直接回答系统在真实生产流量下的服务上限；其二，跨浏览器兼容性覆盖不全，所有功能测试均在 Chrome 上完成，Firefox、Safari、Edge 等主流浏览器尚未形成体系化的对照测试，前端组件在不同内核下的渲染差异与交互一致性也因此缺乏可量化的数据；其三，存储方案与正式生产形态仍有差距，文件存储目前依赖宿主机本地目录持久化，尚未把对象存储作为正式存储路径接入，数据库则使用单实例 MySQL，主从复制与定期备份策略亦未真正落实。上述局限与本课题面向轻量级演示系统的边界相匹配，但若进一步面向企业级生产环境，则仍需在压测、兼容性与高可用层面做专项补强。

在已完成工作的基础上，本课题后续可从三个方向继续推进，整体目标是从“可演示”走向“可运营”。在存储层，可将文件存储抽象为统一接口，分别对接本地目录、MinIO 与公有云对象存储（OSS、COS、OBS），并在元数据中保留文件指纹与多副本路径，以支撑跨存储介质的迁移和冗余备份，移动端图片识别等扩展能力亦可借此叠加；在能力层，可在保留现有 `user_id` 行级隔离机制的基础上，引入团队空间、共享文件夹、协作权限与审批流程，并参考已有 Android 网盘类研究的同步与离线机制完善多端协同访问，使系统从轻量演示原型逐步演进为可服务小型团队的协作式文件管理平台；在工程层，可接入应用性能监控、链路追踪与结构化日志体系，并借助 Prometheus + Grafana 搭建运行指标看板，再以 GitHub Actions 或 GitLab CI 串联镜像构建、单元测试与容器部署，最终把当前的 Docker Compose 部署形态向 Kubernetes 编排平滑迁移，使系统具备从课程作业向小规模生产环境过渡的工程能力。

参考文献

- [1] REIS D, PIEDADE B, CORREIA F F, et al. Developing Docker and Docker-Compose specifications: A developers' survey[J]. IEEE Access, 2022, 10: 2204-2220.
- [2] IBRAHIM M H, SAYAGH M, HASSAN A E. A study of how Docker Compose is used to compose multi-component systems[J]. Empirical Software Engineering, 2021, 26: 128.
- [3] ENG K, HINDLE A, STROULIA E. Patterns of multi-container composition for service orchestration with Docker Compose[J/OL]. arXiv preprint arXiv:2305.11293, 2023.
- [4] PAHL C, BROGI A, SOLDANI J, et al. Cloud container technologies: A state-of-the-art review[J]. IEEE Transactions on Cloud Computing, 2019, 7(3): 677-692.
- [5] 刘鹏. 云计算[M]. 第3版. 北京: 电子工业出版社, 2015.
- [6] YANG P, XIONG N, REN J. Data security and privacy protection for cloud storage: A survey[J]. IEEE Access, 2020, 8: 131723-131740.
- [7] CHEN Y, LI Y, LU Y, et al. Understanding the security risks of websites using cloud storage for direct user file uploads[J]. IEEE Transactions on Information Forensics and Security, 2025, 20: 2677-2691.
- [8] 顾加伟. 云原生环境下容器部署加速策略研究与实现[D]. 南京: 南京邮电大学, 2024.
- [9] 范晓玲. 基于云存储的高职院校网盘系统设计与实现[J]. 河北软件职业技术学院学报, 2017, 19(2): 46-49.
- [10] 胡荣, 杨学玲. 基于 Spring Boot 前后端分离 Web 系统的设计与实现[J]. 信息与电脑, 2023, 35(6): 78-81.
- [11] 游海波. 私有云环境下基于微服务架构的深度学习容器化平台设计与实现[D]. 北京: 北京邮电大学, 2023.
- [12] 朱冠达. 基于云原生架构的微服务治理方案在电力营销系统中的应用研究[D]. 北京: 华北电力大学, 2024.
- [13] 吴亚骏. 基于 MongoDB 的多租户消息推送系统的设计与实现[D]. 北京: 北京邮电大学, 2019.
- [14] DUBOIS P. MySQL[M]. 5th ed. Boston: Addison-Wesley, 2013.
- [15] 黄健宏. Redis 设计与实现[M]. 北京: 机械工业出版社, 2014.
- [16] 李大松. 基于 NodeJS 与云存储的文件管理平台的设计与实现[D]. 武汉: 华中科技大学, 2015.
- [17] 王圆圆. 基于联想云存储智能企业网盘的设计与实现[D]. 北京: 北京邮电大学, 2019.
- [18] Fielding R T. Architectural styles and the design of network-based software architectures[D]. Irvine: University of California, Irvine, 2000.
- [19] 付波. 私有网盘应用系统的设计与实现[D]. 武汉: 华中科技大学, 2018.
- [20] 李华青. 文件在线预览服务快速响应技术研究[D]. 广州: 华南理工大学, 2019.

致 谢

衷心感谢我的指导教师在选题、开题、写作与系统实现各阶段给予的悉心指导。从研究方向的把握、章节结构的安排到论文表述与图表规范的细致审阅，老师严谨的治学态度令我受益匪浅。

感谢本科四年中各位任课老师的悉心教诲，所授课程为本课题的开展奠定了必要的理论与工程基础；也感谢同窗在系统开发与论文写作过程中提供的讨论与反馈，使部分关键环节的实现思路更为清晰。

感谢家人长期以来的理解、关心与支持，这是我能够静心完成本课题的重要支撑。

最后，感谢相关文献、开源项目与技术社区中无私分享经验的研究者与开发者，他们的成果为本文提供了重要的方法论与工程参考。限于作者水平，文中难免存在疏漏，恳请各位老师与评审专家批评指正。